

The Python Data & App Stack

ZERO TO HERO

NumPy

Pandas

Flask

Streamlit

A complete study book — fundamentals, internals, production practice, and interview preparation.

Six parts · 81 chapters · an end-to-end capstone project

· 180+ interview questions with model answers.

Prepared for Sreyanshu

Table of Contents

Introduction and Setup	4
How to Use This Book	4
Environment Setup	5
Roadmap	5
PART I - NumPy	6
1. The Mental Model	6
2. Creating Arrays	7
3. Data Types (dtype)	7
4. Indexing and Slicing	8
5. Broadcasting	9
6. Universal Functions and Vectorization	9
7. Aggregations and the Axis Argument	10
8. Reshaping, Stacking, Splitting	11
9. Linear Algebra	12
10. Random Numbers (Modern API)	12
11. Sorting, Searching, Set Operations	13
12. Performance and Memory	13
13. Structured Arrays and Masked Arrays	14
14. NumPy Gotchas Cheat Sheet	14
15. NumPy Interview Drill	14
PART II - Pandas	16
16. The Mental Model	16
17. Reading and Writing Data	17
18. Selection: [], .loc, .iloc, .at, .iat	17
19. SettingWithCopyWarning - Understood Properly	18
20. Missing Data	19
21. Data Types and Memory	20
22. Creating and Transforming Columns	20
23. GroupBy - Split, Apply, Combine	21
24. Merging, Joining, Concatenating	22
25. Reshaping: pivot, melt, stack, unstack	24
26. String Operations	24
27. Dates and Time Series	25
28. MultiIndex	26
29. Performance and Idioms	26
30. Pandas Gotchas Cheat Sheet	27
31. Pandas Interview Drill	27
PART III - Flask	30
32. The Mental Model	30
33. Routing	31
34. The Request Object	32
35. Responses	32
36. Templates (Jinja2)	33
37. Application and Request Contexts	34

38. Configuration	35
39. The Application Factory and Blueprints	36
40. Databases with Flask-SQLAlchemy	37
41. Building a REST API	39
42. Sessions, Cookies, and Authentication	40
43. Error Handling and Logging	42
44. Hooks and Middleware	43
45. CORS, Rate Limiting, Security	43
46. Testing	44
47. Async, Background Jobs, and Concurrency	45
48. Deployment	45
49. Flask vs Django vs FastAPI	46
50. Flask Interview Drill	47
PART IV - Streamlit	48
51. The Mental Model	48
52. Display Elements	49
53. Widgets	49
54. Session State	50
55. Caching - cache_data vs cache_resource	51
56. Layout	53
57. Charts and Data Display	54
58. Forms	54
59. Multipage Apps and Navigation	55
60. Fragments, Reruns, and Performance	55
61. Secrets, Connections, Config, and Theming	56
62. Deployment	57
63. Streamlit vs Flask vs Dash vs Gradio	58
64. Streamlit Gotchas Cheat Sheet	58
65. Streamlit Interview Drill	58
PART V - Capstone: One Project, All Four Libraries	60
66. What You Are Building	60
67. Stage 1 - The ETL Layer (NumPy + Pandas)	61
68. Stage 2 - The Flask API	63
69. Stage 3 - The Streamlit Dashboard	66
70. Stage 4 - Tests	68
71. Running and Shipping It	69
PART VI - Interview Preparation	71
72. How These Interviews Are Actually Structured	71
73. Rapid-Fire Bank: Python Fundamentals	71
74. Rapid-Fire Bank: NumPy (30 questions)	72
75. Rapid-Fire Bank: Pandas (45 questions)	73
76. Rapid-Fire Bank: Flask (40 questions)	74
77. Rapid-Fire Bank: Streamlit (25 questions)	75
78. Scenario and Design Questions	75
79. Cheat Sheets	76
80. A 30-Day Study Plan	78
81. Closing Notes	78

Introduction and Setup

How to Use This Book

This book is built for two audiences at once: the person learning these libraries for the first time, and the person who already writes code with them but is about to sit in front of an interview panel.

Every part follows the same shape:

1. **Mental model** - what the library actually *is*, and what problem it was built to solve. Skipping this is the single biggest reason people plateau.
2. **Core mechanics** - the 20% of the API that covers 80% of real work, explained with runnable code.
3. **Depth** - the mechanics underneath (memory layout, execution model, context handling) that separate a user from an engineer.
4. **Gotchas** - the bugs everyone writes once. Learn them here instead.
5. **Interview drill** - questions that actually get asked, with model answers and the reasoning behind them.

Notation used throughout

Marker	Meaning
>>>	Interactive interpreter input
# ->	The value an expression evaluates to
GOTCHA	A trap that produces silent wrong answers
INTERVIEW	Frequently asked; know this cold
PERF	Performance-relevant detail

Suggested study order

If you are starting from zero, read in order. NumPy underpins Pandas; Pandas feeds both Flask APIs and Streamlit dashboards. The capstone in Part V ties all four together into one deployable project, which is also the single best thing you can put on a resume for a data/backend hybrid role.

If you are cramming for an interview in under a week, jump to Part VI (the question bank), then return to whichever sections you failed.

Environment Setup

Install

```
# Create an isolated environment (do this per project, always)
python -m venv .venv

# Activate it
source .venv/bin/activate      # macOS / Linux
.venv\Scripts\activate        # Windows PowerShell

# Install the stack
pip install numpy pandas flask streamlit
pip install jupyterlab matplotlib requests python-dotenv
pip install pytest flask-sqlalchemy flask-migrate pyarrow

# Freeze exact versions for reproducibility
pip freeze > requirements.txt
```

Verify

```
import numpy as np, pandas as pd, flask, streamlit as st
print(np.__version__, pd.__version__, flask.__version__, st.__version__)
```

Why virtual environments matter (asked in interviews)

A virtualenv gives each project its own `site-packages` directory. Without it, two projects needing `pandas==1.5` and `pandas==2.2` cannot coexist. The interview answer to "how do you manage dependencies?" should mention: isolation per project, a pinned `requirements.txt` (or `pyproject.toml` with a lock file via Poetry/uv), and a matching Python version recorded somewhere (`.python-version`, Dockerfile, or `pyproject.toml`).

Roadmap

Part	Topic	What you will be able to do
I	NumPy	Vectorize numerical work, reason about memory and broadcasting
II	Pandas	Clean, join, group, reshape, and time-series any tabular data
III	Flask	Build, secure, test, and deploy a production REST API
IV	Streamlit	Ship an interactive data app without front-end code
V	Capstone	Wire all four into one end-to-end project
VI	Interview	Answer the 200 questions that actually get asked

PART I - NumPy

1. The Mental Model

What NumPy actually is

NumPy gives Python one thing it does not have natively: a **contiguous, homogeneous, fixed-type block of memory** with fast C loops over it. Everything else - broadcasting, slicing, linear algebra - is built on that one idea.

A Python list of a million integers is a million pointers to a million separate `PyObject` boxes scattered across the heap. Each box carries a type tag, a reference count, and the actual value. Iterating it means chasing pointers and re-checking types on every single element.

A NumPy array of a million `int64` values is **one** allocation of 8,000,000 contiguous bytes plus a small header. The loop over it is a tight C loop with no type checks, and it is friendly to CPU cache lines and SIMD instructions.

```
import numpy as np, sys

py_list = list(range(1_000_000))
np_arr = np.arange(1_000_000)

sys.getsizeof(py_list)    # ~8,000,056 bytes of POINTERS ALONE
np_arr.nbytes             # 8,000,000 bytes, total, values included
```

INTERVIEW - "Why is NumPy faster than a Python list?" Four reasons, in order of importance:

1. **Contiguous homogeneous memory** - no pointer chasing, cache-friendly.
2. **No per-element type dispatch** - the dtype is known once, up front.
3. **C-level loops** - the interpreter overhead per element disappears.
4. **Vectorization / SIMD** - the compiler can process several elements per CPU instruction.

Say all four. Most candidates say "it's written in C" and stop, which is the weakest of the four answers.

The three attributes that define an array

Every `ndarray` is: **a buffer of bytes + a dtype + a shape + strides.**

```
a = np.arange(12).reshape(3, 4)
a.shape      # (3, 4)      how many elements along each axis
a.dtype      # int64       how to interpret each chunk of bytes
a.strides    # (32, 8)     how many BYTES to step to move one index
a.ndim       # 2
a.size       # 12
a.itemsize   # 8
a.nbytes     # 96
```

Strides are the secret. `a.strides == (32, 8)` means: to move down one row, jump 32 bytes (4 elements x 8 bytes); to move right one column, jump 8 bytes. Almost every "free" operation in NumPy - transposing, slicing, reshaping - is implemented by **rewriting shape and strides without touching the data.**

```
a.T.strides    # (8, 32)   transpose just swaps the strides. Zero copying.
```

2. Creating Arrays

```
np.array([1, 2, 3])           # from a sequence
np.array([[1, 2], [3, 4]])    # 2-D from nested sequences

np.zeros((2, 3))              # filled with 0.0
np.ones((2, 3), dtype=np.int32)
np.full((2, 2), 7)
np.empty((2, 2))              # UNINITIALIZED garbage - fast, be careful
np.eye(3)                     # identity matrix
np.diag([1, 2, 3])            # diagonal matrix

np.arange(0, 10, 2)           # -> [0 2 4 6 8] (step-based, like range)
np.linspace(0, 1, 5)          # -> [0. 0.25 0.5 0.75 1.] (count-based)
np.logspace(0, 3, 4)          # -> [1. 10. 100. 1000.]

np.zeros_like(a)              # same shape AND dtype as `a`
np.ones_like(a); np.empty_like(a)
```

GOTCHA - arange with floats. `np.arange(0, 1, 0.1)` may give 10 or 11 elements depending on floating-point rounding. Use `linspace` whenever the *number of points* matters.

INTERVIEW - arange vs linspace: `arange` takes a step and the endpoint is exclusive; `linspace` takes a count and the endpoint is inclusive by default (`endpoint=False` to change that). Use `linspace` for floats.

3. Data Types (dtype)

```
np.array([1, 2, 3]).dtype     # int64 (platform-dependent; int32 on Win)
np.array([1.0, 2.0]).dtype     # float64
np.array([1, 2], dtype=np.float32)
np.array([1, 2, 3.5])         # UPCAST -> float64, all of it
np.array([1, 'a'])             # UPCAST -> '<U21' strings. Usually a bug.

arr.astype(np.int32)           # explicit cast, always returns a COPY
```

Common dtypes: `int8/16/32/64`, `uint8/.../64`, `float16/32/64`, `complex64/128`, `bool_`, `object_`, `str_ (<U n)`, `datetime64`, `timedelta64`.

Why dtype matters

```
np.array([300], dtype=np.int8) # -> array([44], dtype=int8) SILENT OVERFLOW
np.float32(0.1) + np.float32(0.2) # precision loss is larger than with float64
```

PERF - halving the dtype halves the memory and roughly doubles the number of elements per cache line. For a 10 GB dataset, `float64 -> float32` is often the single highest-leverage optimization available.

INTERVIEW - "How would you reduce the memory of a large numeric array?" Downcast dtypes to the smallest type that represents the value range (`np.iinfo` / `np.finfo` to check limits), use `uint` when there are no negative values, and consider memory-mapping (`np.memmap`) if the data exceeds RAM.

4. Indexing and Slicing

Basic slicing returns a VIEW

```
a = np.arange(10)
b = a[2:5]          # VIEW - shares the same memory buffer
b[0] = 99
a                   # -> [ 0  1 99  3  4  5  6  7  8  9]  `a` CHANGED
```

This is the most important behavioural difference from Python lists, where `lst[2:5]` copies.

```
b.base is a         # True -> b is a view onto a
a[2:5].copy()        # take an explicit copy when you need independence
np.shares_memory(a, b) # the reliable check
```

GOTCHA - a function that slices its input and mutates the slice will mutate the caller's array. Defensive copying (`arr = np.asarray(arr).copy()`) is sometimes worth it at API boundaries.

Multi-dimensional indexing

```
a = np.arange(12).reshape(3, 4)
a[1, 2]          # single element: row 1, col 2 -> 6
a[1]             # whole row 1 -> [4 5 6 7]
a[:, 1]          # whole column 1 -> [1 5 9]
a[0:2, 1:3]      # sub-block
a[::-1]          # reverse rows
a[..., 0]        # Ellipsis: first element of the LAST axis, any ndim
a[np.newaxis, :] # add an axis -> shape (1, 3, 4). Same as a[None, :]
```

Fancy (integer array) indexing returns a COPY

```
a = np.arange(10)
idx = [0, 3, 5]
b = a[idx]        # COPY, not a view
b[0] = 99         # `a` is untouched

# 2-D: index arrays are paired element-wise
m = np.arange(12).reshape(3, 4)
m[[0, 2], [1, 3]] # -> [m[0,1], m[2,3]] = [1, 11]
m[[0, 2][:, [1, 3]] # a 2x2 sub-block (note the two-step form)
np.ix_             # helper: m[np.ix_([0,2],[1,3])] gives the block directly
```

Boolean masking returns a COPY

```
a = np.arange(10)
mask = a % 2 == 0
a[mask]          # -> [0 2 4 6 8]
a[a > 5]          # -> [6 7 8 9]
a[(a > 2) & (a < 7)] # use & | ~ , NOT and/or/not
```

GOTCHA - `and`, `or`, `not` call `__bool__` on the whole array and raise "The truth value of an array with more than one element is ambiguous." Use the bitwise operators, and **parenthesise each condition** because `&` binds tighter than `>`.

```
a[a > 5] = 0           # assignment through a mask works in place
np.where(a > 5, a, 0)   # the functional equivalent, returns a new array
```

INTERVIEW - view vs copy, in one table:

Operation	Result
<code>a[2:5]</code> , <code>a[:,2]</code> , <code>a.T</code> , <code>a.reshape(...)</code> (when possible), <code>a.ravel()</code>	View
<code>a[[0,2,4]]</code> (fancy), <code>a[a>3]</code> (boolean), <code>a.flatten()</code> , <code>a.astype()</code> , <code>a.copy()</code>	Copy

Memorize `ravel` vs `flatten`: `ravel()` returns a view when it can, `flatten()` always copies.

5. Broadcasting

The rule that unlocks most of NumPy. Two arrays are compatible when, comparing their shapes **right-aligned**, each dimension pair is either equal, or one of them is 1.

```
A      (3, 4)      A      (3, 1)      A      (3, 4)
B      (4,)   OK   B      (4,)   OK   B      (2, 4)   ERROR
result (3, 4)      result (3, 4)      (3 vs 2, neither is 1)
```

```
a = np.arange(12).reshape(3, 4)
a + 10           # scalar broadcast over everything
a + np.array([0, 1, 2, 3]) # (3,4) + (4,) -> row vector added to every row
a + np.array([[0], [1], [2]]) # (3,4) + (3,1) -> column vector per row

# The classic: pairwise distance / outer operations
x = np.array([1, 2, 3])
x[:, None] - x[None, :] # (3,1) - (1,3) -> (3,3) difference matrix
```

Broadcasting does **not** allocate the expanded array. It is implemented with a stride of 0 along the broadcast axis, so the same memory is read repeatedly.

GOTCHA - the shape (n,) vs (n,1) trap. A "row" of a DataFrame or the result of `arr.sum(axis=1)` is 1-D with shape `(n,)`. Subtracting it from an `(n, n)` matrix broadcasts along the *wrong* axis unless you reshape to `(n, 1)`. This is the number-one source of silent numerical bugs.

```
X = np.random.rand(100, 5)
# Standardize columns - correct:
(X - X.mean(axis=0)) / X.std(axis=0)           # (100,5) - (5,)   OK
# Standardize ROWS - must reshape:
(X - X.mean(axis=1, keepdims=True)) / X.std(axis=1, keepdims=True)
```

`keepdims=True` is the idiomatic fix. Learn it.

INTERVIEW - "Explain broadcasting." Give the right-alignment rule, then the memory point (no copy, stride-0), then a concrete example like mean-centering.

6. Universal Functions and Vectorization

A ufunc is an elementwise C-loop over arrays: `np.add`, `np.exp`, `np.sqrt`, `np.maximum`, `np.sin`, and dozens more. Operators like `+` dispatch to them.

```

np.sqrt(a); np.exp(a); np.log(a + 1); np.abs(a)
np.maximum(a, b)           # elementwise max of TWO arrays
np.clip(a, 0, 10)
np.round(a, 2); np.floor(a); np.ceil(a)

# Ufunc methods
np.add.reduce(a)           # same as a.sum()
np.add.accumulate(a)      # same as a.cumsum()
np.add.outer([1,2], [10,20]) # -> [[11,21],[12,22]]
np.multiply.reduceat(a, [0, 3, 7]) # segmented reduction

```

In-place operations and `out=`

```

a += 1                     # in place, no new allocation
np.exp(a, out=a)          # write result back into `a`
np.add(a, b, out=result)  # reuse a preallocated buffer

```

PERF - inside a hot loop, `out=` and `+=` avoid allocating a new array each iteration. On large arrays this can be a 2x win.

`np.vectorize` is NOT vectorization

```

f = np.vectorize(lambda x: x * 2 + 1) # a convenience wrapper, a Python LOOP

```

INTERVIEW - if asked about `np.vectorize`, say clearly: it provides broadcasting semantics and a nicer API, but it does **not** give a speedup - it loops in Python. For real speed use native ufuncs, or `numba`/`cython`.

7. Aggregations and the Axis Argument

```

a = np.arange(12).reshape(3, 4)
a.sum()           # 66 - over everything
a.sum(axis=0)     # shape (4,) - collapses ROWS, one value PER COLUMN
a.sum(axis=1)     # shape (3,) - collapses COLUMNS, one value PER ROW
a.sum(axis=0, keepdims=True) # shape (1, 4)

```

The rule that makes `axis` click: `axis=k` means "the *k*-th axis disappears from the result." `axis=0` on a (3,4) array removes the 3, leaving (4,). Do not memorize "axis 0 is columns" - that phrasing breaks in 3-D.

```

a.mean(); a.std(); a.var(); a.min(); a.max(); a.prod()
a.argmin(); a.argmax()           # INDEX of the extreme value
np.median(a); np.percentile(a, [25, 50, 75]); np.quantile(a, 0.9)
a.cumsum(); a.cumprod()
np.unique(a, return_counts=True)
a.any(); a.all()
np.count_nonzero(a > 5)

```

NaN-aware versions

```
b = np.array([1.0, np.nan, 3.0])
b.sum()           # nan - NaN poisons everything
np.nansum(b)      # 4.0
np.nanmean(b); np.nanmax(b); np.nanstd(b)
np.isnan(b)       # the ONLY correct NaN test
b == np.nan       # ALWAYS False. NaN != NaN by IEEE-754.
```

INTERVIEW - "How do you check for NaN?" `np.isnan()` / `pd.isna()`. Explain why: NaN compares unequal to everything including itself, so `== np.nan` silently returns all-False.

`np.std` uses `ddof=0` (population) by default, while pandas `.std()` uses `ddof=1` (sample). This discrepancy is a favourite interview trick question.

8. Reshaping, Stacking, Splitting

```
a = np.arange(12)
a.reshape(3, 4)
a.reshape(3, -1)      # -1 = "infer this dimension"
a.reshape(-1, 1)      # column vector
a.ravel()             # flatten to 1-D, VIEW when possible
a.flatten()           # flatten to 1-D, always a COPY
a.T                   # transpose (view)
np.transpose(a3d, (2, 0, 1)) # arbitrary axis permutation
np.swapaxes(a, 0, 1); np.moveaxis(a, 0, -1)
np.squeeze(x)         # drop axes of length 1
np.expand_dims(x, 0)  # add an axis
```

C order vs Fortran order

`reshape` and `ravel` take `order='C'` (row-major, default, last axis varies fastest) or `order='F'` (column-major). `a.flags['C_CONTIGUOUS']` tells you the current layout. A non-contiguous array cannot always be reshaped as a view, in which case NumPy silently makes a copy.

```
np.ascontiguousarray(a)      # force C-contiguous, for interop with C libs
```

Joining and splitting

```
np.concatenate([a, b], axis=0)      # the general workhorse
np.vstack([a, b])                   # stack rows (axis 0)
np.hstack([a, b])                   # stack columns (axis 1 for 2-D, 0 for 1-D)
np.stack([a, b], axis=0)             # NEW axis - shapes must match exactly
np.column_stack([x, y])             # 1-D arrays become columns
np.tile(a, (2, 3)); np.repeat(a, 3)

np.split(a, 3)                      # equal parts, errors if not divisible
np.array_split(a, 3)                # allows unequal parts
np.hsplit(a, 2); np.vsplit(a, 2)
```

INTERVIEW - **`np.stack` vs `np.concatenate`**: `concatenate` joins along an *existing* axis (dimensionality unchanged); `stack` creates a *new* axis (dimensionality +1). `np.tile` repeats the whole array; `np.repeat` repeats each element.

PERF - never build an array by repeated `np.append` or `np.concatenate` in a loop. Each call reallocates and copies the whole thing, giving $O(n^2)$ behaviour. Collect into a Python list and call `np.array(lst)` once, or preallocate with `np.empty(n)` and fill by index.

9. Linear Algebra

```
A = np.array([[1., 2.], [3., 4.]])
b = np.array([5., 6.])

A @ A          # matrix multiply (preferred, Python 3.5+)
np.dot(A, A)    # same for 2-D
A * A          # ELEMENTWISE - a very common confusion
np.matmul(A, A) # like @, broadcasts over stacks of matrices

np.linalg.inv(A)
np.linalg.det(A)
np.linalg.matrix_rank(A)
np.trace(A)
np.linalg.solve(A, b)      # solve Ax = b - USE THIS, not inv(A) @ b
np.linalg.eig(A)           # eigenvalues, eigenvectors
np.linalg.svd(A)           # singular value decomposition
np.linalg.norm(b)          # L2 norm; ord= for others
np.linalg.lstsq(A, b, rcond=None) # least squares
np.einsum('ij,jk->ik', A, A) # Einstein summation: the power tool
```

INTERVIEW - why solve over inv? Computing an explicit inverse is slower (roughly 2-3x the flops) and numerically less stable. `solve` uses an LU factorization directly. The same applies in any numerical library, so this answer travels well.

INTERVIEW - * vs @: `*` is Hadamard (elementwise) product, `@` is matrix multiplication. On 1-D arrays, `@` is the dot product.

10. Random Numbers (Modern API)

```
rng = np.random.default_rng(seed=42)      # the modern Generator API

rng.random(5)                            # uniform [0, 1)
rng.integers(0, 10, size=5)               # ints in [0, 10)
rng.normal(loc=0, scale=1, size=(3, 3))
rng.choice([1,2,3], size=5, replace=False, p=[0.2, 0.3, 0.5])
rng.shuffle(a)                            # in place
rng.permutation(a)                        # returns a shuffled COPY
```

The legacy API (`np.random.seed`, `np.random.rand`) still works but uses one global mutable state, which is a problem for reproducibility across threads and libraries. **INTERVIEW answer:** prefer `default_rng` with an explicit seed; pass the generator object around rather than relying on global state.

Note the API differences: `rng.integers` is high-exclusive; legacy `np.random.randint` is also high-exclusive, but `np.random.random_integers` (deprecated) was inclusive.

11. Sorting, Searching, Set Operations

```
np.sort(a)           # returns a sorted COPY
a.sort()             # sorts IN PLACE, returns None
np.argsort(a)        # indices that would sort a
a[np.argsort(a)]      # sorted, via indices

# Sort a 2-D array BY a column - very common interview task
m = np.array([[3, 1], [1, 2], [2, 0]])
m[m[:, 0].argsort()]  # sort rows by the first column

np.lexsort((secondary, primary))  # multi-key sort (LAST key is primary)
np.partition(a, 3)                 # O(n) partial sort: first 3 are the 3 smallest
np.argpartition(a, -5)[-5:]        # indices of the 5 largest, O(n) not O(n log n)

np.searchsorted(sorted_a, 5)       # binary search insertion point
np.where(cond, x, y)               # vectorized ternary
np.where(a > 5)                    # with ONE argument: indices where True
np.nonzero(a)                      # multi-branch where
np.select([c1, c2], [v1, v2], default=0)

np.unique(a); np.intersect1d(a, b); np.union1d(a, b)
np.setdiff1d(a, b); np.isin(a, [1, 2, 3])
```

INTERVIEW - "Find the top-k elements efficiently." `np.argpartition` is $O(n)$ versus `np.argsort`'s $O(n \log n)$. Knowing this distinction signals real familiarity.

12. Performance and Memory

```
%timeit np.sum(a)      # in Jupyter
a.nbytes               # actual byte usage
np.shares_memory(a, b) # view detection
a.flags                # contiguity, writeability, ownership
```

Checklist for making NumPy code fast:

1. **Remove Python loops.** One vectorized expression beats any `for` loop.
2. **Avoid repeated allocation.** Use `out=`, `+=`, or preallocate.
3. **Downcast dtypes** when the precision is not needed.
4. **Respect memory order.** Iterating a C-contiguous array along the last axis is cache-friendly; along the first axis it is not.
5. **Avoid `dtype=object`** arrays - they are lists wearing a costume, with all the pointer chasing back.
6. **Use `np.memmap`** for datasets larger than RAM.
7. **Reach for `numba`, `cython`, or `numexpr`** only after the above.

```
# memory-map a large file instead of loading it
mm = np.memmap('big.dat', dtype='float32', mode='r', shape=(1_000_000, 100))

# save / load
np.save('a.npy', a); np.load('a.npy')
np.savez_compressed('many.npz', x=a, y=b)
np.savetxt('a.csv', a, delimiter=','); np.loadtxt('a.csv', delimiter=',')
```

13. Structured Arrays and Masked Arrays

```
dt = np.dtype([('name', 'U10'), ('age', 'i4'), ('score', 'f8')])
people = np.array([('Ana', 30, 91.5), ('Bo', 25, 88.0)], dtype=dt)
people['age'] # field access by name
people[people['age'] > 26]

# Masked arrays: carry missingness explicitly
m = np.ma.masked_array([1, 2, 3], mask=[0, 1, 0])
m.mean() # 2.0 - the masked element is excluded
```

Structured arrays are rarely the right answer in 2026 (use Pandas or PyArrow), but knowing they exist - and that they're the basis of `np.recarray` - is worth a sentence in an interview.

14. NumPy Gotchas Cheat Sheet

Trap	What actually happens	Fix
<code>a[a>5]</code> and <code>a[a<2]</code>	ValueError: ambiguous truth value	use the bitwise operators <code>&</code> , <code>~</code> , and <code>or</code> , with parentheses
<code>x == np.nan</code>	always False	<code>np.isnan(x)</code>
<code>b = a[1:]; b[0]=9</code>	mutates <code>a</code>	<code>.copy()</code>
<code>np.array([300], np.int8)</code>	wraps to 44	check <code>np.iinfo</code> limits
<code>arr.sort()</code> returns None	in-place	use <code>np.sort(arr)</code> for a copy
<code>a + b</code> shape error	broadcasting mismatch	<code>keepdims=True</code> or <code>[:, None]</code>
<code>np.append</code> in a loop	$O(n^2)$	list-then- <code>np.array</code> , or preallocate
<code>std()</code> differs from pandas	<code>ddof=0</code> vs <code>ddof=1</code>	pass <code>ddof</code> explicitly
<code>a.reshape(...)</code> sometimes copies	non-contiguous input	<code>np.ascontiguousarray</code> first
float equality <code>0.1+0.2 == 0.3</code>	False	<code>np.isclose</code> / <code>np.allclose</code>

15. NumPy Interview Drill

Q1. What is an ndarray and how does it differ from a Python list? Fixed-size, homogeneous, contiguous buffer with a dtype, shape, and strides. Lists are dynamic, heterogeneous arrays of pointers. The ndarray enables C loops, cache efficiency, and vectorization; lists cannot support any of these.

Q2. Explain shape, strides, and how transpose is free. Shape counts elements per axis; strides count bytes to advance one index on each axis. `a.T` swaps both, reusing the same buffer, so no data moves.

Q3. When do you get a view and when a copy? Basic slicing and reshape/ravel/transpose give views; fancy indexing, boolean masking, `flatten`, and `astype` give copies. Verify with `np.shares_memory`.

Q4. What are the broadcasting rules? Right-align shapes; each dimension pair must be equal or contain a 1; missing leading dimensions are treated as 1. No data is copied - the broadcast axis gets stride 0.

Q5. `np.dot` vs `np.matmul` vs `*`? `*` is elementwise. For 2-D, `dot` and `matmul` agree. They diverge for >2-D: `matmul` broadcasts stacks of matrices, `dot` performs a sum-product over the last axis of A and the second-to-last of B. `@` is `matmul`.

Q6. How do you handle missing values in NumPy? `np.nan` only exists for float dtypes. Detect with `np.isnan`, aggregate with the `nan*` family, or use `np.ma` masked arrays. For integer missingness you need Pandas' nullable dtypes or a sentinel value.

Q7. How would you normalize each row of a matrix?

```
X / np.linalg.norm(X, axis=1, keepdims=True)
```

The `keepdims=True` is the whole point of the question.

Q8. Find the k largest elements without a full sort.

```
idx = np.argpartition(a, -k)[-k:]  
idx[np.argsort(a[idx])[:, -1]] # order them if needed
```

Q9. How do you compute a pairwise distance matrix without loops?

```
d = np.sqrt(((X[:, None, :] - X[None, :, :]) ** 2).sum(axis=-1))
```

Mention the memory cost: this materializes an `(n, n, d)` intermediate. The memory-efficient identity is $||x-y||^2 = ||x||^2 + ||y||^2 - 2x.y$, computable with one `matmul`.

Q10. What is `np.einsum` good for? Expressing arbitrary contractions, transpositions, and reductions in one readable string, often avoiding intermediates. `np.einsum('ij,jk->ik', A, B)` is `matmul`; `'ii->i'` extracts a diagonal; `'ij->ji'` transposes.

Q11. Why does `0.1 + 0.2 != 0.3`? Binary floating point cannot represent these decimals exactly. Compare with `np.isclose(0.1+0.2, 0.3)` and never test float equality directly.

Q12. What does `-1` mean in `reshape`? Infer that dimension from the total element count. Only one `-1` allowed.

Q13. Difference between `ravel()` and `flatten()`? `ravel` returns a view when the array is contiguous (cheap); `flatten` always copies (safe).

Q14. How do you seed randomness reproducibly? `rng = np.random.default_rng(42)` and pass `rng` explicitly. Avoid the global legacy `np.random.seed`, which is shared mutable state.

Coding exercises (do these on paper first)

- Reverse a 1-D array without `[::-1]`. (Hint: `np.flip`.)
- Replace all negative values with 0 in three different ways.
- Given a 2-D array, subtract the row mean from each row.
- Count how many elements of each row exceed that row's mean.
- Given `a`, produce a boolean array marking the first occurrence of each unique value. (Hint: `np.unique(..., return_index=True)`.)
- Implement one-hot encoding for an integer label array without loops. (Hint: `np.eye(n_classes)[labels]`.)
- Compute a moving average of window 3 using `np.convolve` and again using `cumsum`.
- Find the index of the value closest to a given scalar.
- Given two arrays, return elements present in `a` but not `b`, preserving order.
- Build a checkerboard 8x8 matrix of 0s and 1s in one expression.

PART II - Pandas

16. The Mental Model

Pandas is NumPy plus **labels**. A NumPy array is addressed by position; a Pandas object is addressed by an **Index**. That single addition brings alignment, joins, group-by, time series, and missing-data handling.

Three objects matter:

- **Index** - an immutable, often-hashed array of labels. It powers alignment and O(1) lookups.
- **Series** - a 1-D labelled array: one NumPy (or Arrow) array + one Index.
- **DataFrame** - a dict of Series sharing one row Index, with a second Index for the columns.

```
import pandas as pd, numpy as np

s = pd.Series([10, 20, 30], index=['a', 'b', 'c'], name='sales')
df = pd.DataFrame({
    'name': ['Ana', 'Bo', 'Cy', 'Di'],
    'dept': ['eng', 'eng', 'ops', 'ops'],
    'salary': [100, 120, 90, 95],
    'start': pd.to_datetime(['2021-01-05', '2022-03-01', '2020-07-19', '2023-02-11'])
})
```

Automatic alignment - the thing that surprises newcomers

```
a = pd.Series([1, 2, 3], index=['x', 'y', 'z'])
b = pd.Series([10, 20, 30], index=['z', 'y', 'w'])
a + b
# w      NaN
# x      NaN
# y     22.0
# z     13.0
```

Pandas aligned on labels, not position, and produced NaN where labels did not match. **INTERVIEW:** this is the correct answer to "what happens when you add two Series with different indexes?" - union of indexes, NaN for non-overlap. Use `a.add(b, fill_value=0)` to control it.

Inspecting anything

```
df.head(); df.tail(); df.sample(5)
df.shape; df.columns; df.index; df.dtypes
df.info()           # dtypes + non-null counts + memory: your first command
df.describe()       # numeric summary
df.describe(include='all')
df.memory_usage(deep=True) # deep=True counts the actual string bytes
df.nunique(); df['dept'].value_counts(dropna=False)
```

17. Reading and Writing Data

```
pd.read_csv('f.csv')
pd.read_csv('f.csv',
            usecols=['a','b'],          # read fewer columns -> less memory
            dtype={'id': 'int32'},      # avoid the inference pass
            parse_dates=['start'],
            na_values=['NA','-','?'],
            nrows=1000,                 # peek at a big file
            chunksize=100_000,         # returns an ITERATOR of DataFrames
            sep=';', encoding='utf-8', low_memory=False)

pd.read_excel('f.xlsx', sheet_name='Sheet1')
pd.read_json('f.json'); pd.read_parquet('f.parquet')
pd.read_sql('SELECT * FROM t WHERE x > 5', con=engine)
pd.read_html('https://example.com/page') # returns a LIST of DataFrames
pd.read_clipboard()

df.to_csv('out.csv', index=False)      # index=False is almost always right
df.to_parquet('out.parquet')          # columnar, typed, compressed
df.to_sql('table', con=engine, if_exists='append', index=False)
```

INTERVIEW - "How do you read a file too large for memory?"

1. `usecols` + narrow `dtype` to shrink it first.
2. `chunksize` to stream and aggregate chunk by chunk.
3. Convert to Parquet, which is columnar and compressed, then read only the columns you need.
4. Move to Dask, Polars, or DuckDB for out-of-core processing.

```
total = 0
for chunk in pd.read_csv('huge.csv', chunksize=200_000):
    total += chunk.loc[chunk.amount > 0, 'amount'].sum()
```

CSV vs Parquet: CSV is row-oriented, untyped, and re-parsed on every read. Parquet stores dtypes, compresses per column, and supports column pruning and predicate pushdown. For anything repeated, Parquet wins by a wide margin.

18. Selection: `[]`, `.loc`, `.iloc`, `.at`, `.iat`

This is where beginners lose the most time. Learn the four rules.

```
df['salary']          # a Series (one column)
df[['salary','dept']] # a DataFrame (list of columns)
df[0:2]               # ROWS by position - inconsistent, avoid
df[df.salary > 95]     # rows by boolean mask
```

`df[...]` is overloaded and context-dependent. Prefer the explicit accessors:

```

# .loc -> LABEL based, and the endpoint IS INCLUSIVE
df.loc[0]                # row with index label 0
df.loc[0:2]              # labels 0,1,2 <- three rows, inclusive!
df.loc[df.dept == 'eng', ['name', 'salary']]
df.loc[:, 'name':'salary'] # label slice of columns
df.loc[df.salary > 100, 'bonus'] = 10    # the correct way to assign

# .iloc -> POSITION based, endpoint EXCLUSIVE (like normal Python)
df.iloc[0]
df.iloc[0:2]                # positions 0,1 <- two rows
df.iloc[-1]                 # last row
df.iloc[[0,2], [1,3]]

# .at / .iat -> single scalar, fastest path
df.at[0, 'salary']
df.iat[0, 2]

```

INTERVIEW - .loc vs .iloc in one sentence: `.loc` selects by label and includes the stop label; `.iloc` selects by integer position and excludes it. Add the trap: if your index is integers, `.loc[0:2]` and `.iloc[0:2]` return different numbers of rows.

query and eval

```

df.query('salary > 95 and dept == "eng"')
threshold = 95
df.query('salary > @threshold')           # @ references Python variables
df.eval('total = salary * 1.1', inplace=True)

```

`query/eval` can use `numexpr` to avoid intermediate temporaries on large frames, and they read cleanly. On small frames they are slower due to parsing overhead.

Other selectors

```

df.filter(like='sal')                # columns whose name contains 'sal'
df.filter(regex='^s')
df.select_dtypes(include='number')
df.select_dtypes(exclude=['object', 'category'])
df.nlargest(3, 'salary'); df.nsmallest(3, 'salary')
df.isin({'dept': ['eng']})
df.between_time('09:00', '17:00')    # for a DatetimeIndex

```

19. SettingWithCopyWarning - Understood Properly

```

sub = df[df.dept == 'eng']           # this is (probably) a COPY
sub['salary'] = 0                     # SettingWithCopyWarning

```

The warning says: *you modified something that may or may not be a view of the original, so I cannot tell you whether df changed.* It is a warning about **ambiguity**, not about correctness of your syntax.

The fix is always the same: do the selection and the assignment in one `.loc` call.

```

df.loc[df.dept == 'eng', 'salary'] = 0    # unambiguous, modifies df
sub = df[df.dept == 'eng'].copy()         # or take an explicit copy
sub['salary'] = 0                          # now clearly independent

```

INTERVIEW - "What is chained indexing and why is it bad?" `df[df.a > 1]['b'] = 0` performs two separate `__getitem__` / `__setitem__` operations. The first may return a copy, so the assignment lands on a temporary that is discarded. Use single-step `.loc`. Note also that Pandas 3.0 adopts Copy-on-Write semantics, under which chained assignment *never* propagates - which makes the single-`.loc` habit mandatory rather than merely advisable.

20. Missing Data

```
df.isna(); df.notna()
df.isna().sum()           # nulls per column - your second command
df.isna().mean().sort_values(ascending=False)  # null RATE per column

df.dropna()               # drop rows with ANY null
df.dropna(how='all')       # only fully-empty rows
df.dropna(subset=['salary']) # only when specific columns are null
df.dropna(axis=1, thresh=100) # drop columns with <100 non-nulls

df.fillna(0)
df.fillna({'salary': 0, 'dept': 'unknown'}) # per-column fill
df['salary'].fillna(df['salary'].median())
df.ffill(); df.bfill()      # forward / backward fill
df['x'].interpolate(method='linear') # numeric interpolation
```

The sentinel zoo

Value	Appears in	Detected by
<code>np.nan</code>	float columns	<code>isna()</code>
<code>None</code>	object columns	<code>isna()</code>
<code>pd.NaT</code>	datetime/timedelta	<code>isna()</code>
<code>pd.NA</code>	nullable Int64/boolean/string	<code>isna()</code>

INTERVIEW - "Why does an integer column become float after you introduce a missing value?" Because `np.nan` is a float, and NumPy integer dtypes have no NaN representation, so the column upcasts to `float64`. The modern fix is the nullable dtype `Int64` (capital I), which uses a separate mask.

```
s = pd.Series([1, 2, None], dtype='Int64') # stays integer, holds pd.NA
```

Imputation strategy talking points: mean (sensitive to outliers), median (robust, default choice for skewed numerics), mode (categoricals), forward-fill (time series where the last observation carries), group-wise fill (`df.groupby('dept')['salary'].transform(lambda s: s.fillna(s.median()))`), and model-based imputation. Critically: **fit imputation on the training set only**, otherwise you leak information.

21. Data Types and Memory

```
df.dtypes
df['dept'] = df['dept'].astype('category')      # huge win for low-cardinality
df['id']    = pd.to_numeric(df['id'], downcast='integer')
df['start'] = pd.to_datetime(df['start'], errors='coerce') # bad values -> NaT
df['flag']  = df['flag'].astype('boolean')       # nullable boolean
df['name']  = df['name'].astype('string')        # nullable StringDtype
```

Categorical dtype stores an integer code array plus a small dictionary of categories. A column of 1,000,000 values drawn from 5 distinct strings drops from tens of megabytes to about one.

```
def shrink(df):
    for c in df.select_dtypes('int').columns:
        df[c] = pd.to_numeric(df[c], downcast='integer')
    for c in df.select_dtypes('float').columns:
        df[c] = pd.to_numeric(df[c], downcast='float')
    for c in df.select_dtypes('object').columns:
        if df[c].nunique() / len(df) < 0.5:
            df[c] = df[c].astype('category')
    return df
```

INTERVIEW - "How do you reduce a DataFrame's memory footprint?" Downcast numerics, convert low-cardinality strings to `category`, drop unused columns at read time with `usecols`, use `chunksize`, and store as Parquet. Measure with `df.memory_usage(deep=True)` - `deep=True` matters because object columns otherwise report only pointer sizes.

22. Creating and Transforming Columns

```
df['bonus'] = df['salary'] * 0.1                # vectorized, fast
df['band']  = np.where(df.salary > 100, 'high', 'low')
df['band']  = pd.cut(df.salary, bins=[0, 95, 110, np.inf],
                    labels=['low', 'mid', 'high'])
df['decile'] = pd.qcut(df.salary, q=10, labels=False) # equal-FREQUENCY bins

df = df.assign(total=lambda d: d.salary + d.bonus,
               ratio=lambda d: d.total / d.total.sum())
```

`assign` returns a new frame and allows chaining, which keeps pipelines readable and side-effect-free.

apply vs map vs applymap vs vectorized

```
df['salary'].map(lambda x: x * 2)                # Series -> elementwise
df['dept'].map({'eng': 'Engineering'})           # Series -> dictionary lookup
df.apply(lambda row: row.salary * 2, axis=1)     # DataFrame -> row-wise, SLOW
df.apply(np.sum, axis=0)                         # DataFrame -> column-wise
df.map(lambda x: x)                              # elementwise on whole DataFrame
# (was applymap before Pandas 2.1)
```

INTERVIEW - "**apply vs vectorized operations**": `apply` runs a Python callable once per row or column - it is a loop with nicer syntax, typically 10-100x slower than the vectorized equivalent. Always try, in this order: (1) a native vectorized expression, (2) `np.where` / `np.select` for branching, (3) `map` with a dict for lookups, (4) `apply` only as a last resort. Mention that `axis=1` `apply` is the slowest form because it constructs a Series per row.

```
# Bad
df['tax'] = df.apply(lambda r: r.salary*0.3 if r.salary>100 else r.salary*0.2,
                    axis=1)

# Good
df['tax'] = np.where(df.salary > 100, df.salary*0.3, df.salary*0.2)

# Multi-branch
df['tier'] = np.select([df.salary > 110, df.salary > 95],
                      ['A', 'B'], default='C')
```

Other column operations

```
df.rename(columns={'name':'employee'})
df.columns = df.columns.str.lower().str.replace(' ', '_')
df.drop(columns=['bonus'])
df.drop_duplicates(subset=['name'], keep='first')
df.duplicated().sum()
df.sort_values(['dept','salary'], ascending=[True, False])
df.sort_index()
df.reset_index(drop=True)
df.set_index('name')
df.replace({'eng':'engineering'})
df.clip(lower=0, upper=200)
df.round(2)
df.rank(method='dense', ascending=False)
```

23. GroupBy - Split, Apply, Combine

The single most-tested Pandas concept.

```
g = df.groupby('dept')

g['salary'].mean()
g.agg({'salary': ['mean','max'], 'name': 'count'})
g.size() # rows per group (includes NaN)
g.count() # non-null count per column
g.first(); g.last(); g.nth(0)
g.describe()
```

Named aggregation (the clean modern form)

```
df.groupby('dept').agg(
    avg_salary=('salary', 'mean'),
    top_salary=('salary', 'max'),
    headcount=('name', 'size'),
    spread=('salary', lambda s: s.max() - s.min()),
)
```

agg VS transform VS filter VS apply

Method	Returns	Use for
agg	one row per group	summaries
transform	same shape as input	broadcasting a group stat back onto rows
filter	subset of original rows	keeping/dropping whole groups
apply	anything	when nothing else fits (slowest)

```
# transform: add each employee's deviation from their department mean
df['dept_mean'] = df.groupby('dept')['salary'].transform('mean')
df['vs_dept'] = df.salary - df.dept_mean

# filter: keep only departments with more than 1 person
df.groupby('dept').filter(lambda g: len(g) > 1)

# apply: arbitrary per-group logic
df.groupby('dept').apply(lambda g: g.nlargest(2, 'salary'))
```

INTERVIEW - "Difference between agg and transform?" `agg` collapses each group to a single value (result is indexed by group); `transform` returns a result aligned to the *original* index, so you can assign it straight back as a column. This is the pandas equivalent of a SQL window function.

More groupby patterns

```
df.groupby(['dept', 'band'])['salary'].sum()           # multi-key -> MultiIndex
df.groupby('dept', as_index=False)['salary'].mean()   # keep dept as a column
df.groupby('dept', dropna=False)                      # include NaN groups
df.groupby('dept', observed=True)                     # categoricals: skip empties
df.groupby('dept')['salary'].rank(ascending=False)     # rank within group
df.groupby('dept').cumsum()
df.groupby(df.start.dt.year)['salary'].mean()         # group by a derived key
df.groupby(pd.Grouper(key='start', freq='ME'))        # group by month-end
```

PERF - `groupby().apply()` with a Python function is slow because it materializes each group. Prefer built-in string aggregations (`'mean'`, `'sum'`) which dispatch to Cython, and prefer `transform('mean')` over `transform(lambda s: s.mean())` for the same reason.

24. Merging, Joining, Concatenating

```
pd.merge(left, right, on='key', how='inner')
pd.merge(left, right, left_on='id', right_on='user_id', how='left')
pd.merge(left, right, left_index=True, right_index=True)
left.join(right, how='left')                          # index-based convenience wrapper
pd.concat([df1, df2], axis=0, ignore_index=True)       # stack rows
pd.concat([df1, df2], axis=1)                         # align on index, side by side
```

Join types

how	Keeps
inner	only keys present in both (default)
left	all left rows; NaN where right is missing
right	all right rows
outer	union of keys
cross	Cartesian product

The parameters that save you from silent bugs

```
pd.merge(a, b, on='k', how='left',
         suffixes=('_left', '_right'), # disambiguate colliding column names
         indicator=True,               # adds _merge: left_only/right_only/both
         validate='one_to_many')      # RAISES if the relationship is violated
```

INTERVIEW - "Your merge produced more rows than the left frame. Why?" Duplicate keys on the right side: a left join fans out one left row into n matches. Diagnose with `right['k'].duplicated().sum()` and prevent with `validate='m:1'`. This question separates people who have shipped data pipelines from people who have not.

```
# Verify a join did what you think
m = pd.merge(a, b, on='k', how='left', indicator=True)
m['_merge'].value_counts() # how many left_only rows failed to match?
```

merge_asof - nearest-key joins (time series)

```
pd.merge_asof(trades.sort_values('time'),
              quotes.sort_values('time'),
              on='time', by='symbol',
              direction='backward',
              tolerance=pd.Timedelta('2ms'))
```

Joins each trade to the most recent preceding quote. It is the canonical answer to "how do you join two time series that don't share exact timestamps?"

25. Reshaping: pivot, melt, stack, unstack

```
# WIDE <- LONG
df.pivot(index='date', columns='city', values='temp')           # no aggregation
df.pivot_table(index='dept', columns='band', values='salary',
               aggfunc='mean', fill_value=0, margins=True)      # aggregates dupes

# LONG <- WIDE
df.melt(id_vars=['name'], value_vars=['q1', 'q2'],
       var_name='quarter', value_name='amount')

# index/column juggling
df.stack()              # columns -> inner index level (wide to long)
df.unstack()            # inner index level -> columns (long to wide)
df.T

pd.crosstab(df.dept, df.band, normalize='index', margins=True)
pd.get_dummies(df, columns=['dept'], drop_first=True)          # one-hot encoding
```

INTERVIEW - pivot vs pivot_table: `pivot` merely reshapes and raises on duplicate index/column pairs; `pivot_table` aggregates duplicates using `aggfunc` (default `mean`) and can add margins. `melt` is the inverse of both.

Tidy data (worth naming in interviews): each variable is a column, each observation is a row, each observational unit is a table. `melt` moves data *toward* tidy; `pivot` moves it toward a human-readable report.

26. String Operations

```
s = df['name'].str
s.lower(); s.upper(); s.title(); s.strip()
s.len(); s.contains('an', case=False, na=False)
s.startswith('A'); s.endswith('o')
s.replace('a', 'A', regex=False)
s.split(' ', expand=True)           # expand=True -> a DataFrame of parts
s.extract(r'(\d+)-(\d+)')           # regex capture groups -> columns
s.extractall(r'(\d+)')
s.findall(r'\d+')
s.cat(sep=', ')                    # join all values into one string
s.pad(10, side='left', fillchar='0'); s.zfill(5)
s.slice(0, 3); s.get(0)
s.normalize('NFKD')
```

GOTCHA - `.str.contains` returns `NaN` for missing values, which then breaks boolean indexing with "Cannot mask with non-boolean array". Always pass `na=False`.

PERF - `.str` methods still loop in Python under the object dtype. Convert to `string[pyarrow]` dtype for a substantial speedup on large text columns: `df['name'] = df['name'].astype('string[pyarrow]')`.

27. Dates and Time Series

```
pd.to_datetime(df['start'], format='%Y-%m-%d', errors='coerce')
pd.date_range('2024-01-01', periods=12, freq='ME')      # month end
pd.date_range('2024-01-01', '2024-12-31', freq='W-MON')

d = df['start'].dt
d.year; d.month; d.day; d.hour; d.dayofweek; d.day_name()
d.quarter; d.is_month_end; d.days_in_month
d.to_period('M'); d.normalize(); d.floor('h'); d.round('15min')
d.tz_localize('UTC').dt.tz_convert('Asia/Kolkata')

df['start'] - pd.Timestamp('2024-01-01')      # Timedelta
df['start'] + pd.DateOffset(months=3)          # calendar-aware offset
```

Resampling and rolling windows

```
ts = df.set_index('start').sort_index()

ts.resample('ME')['salary'].sum()              # downsample: monthly totals
ts.resample('D').ffill()                      # upsample: fill forward
ts.resample('W').agg({'salary': ['sum', 'mean']})

ts['salary'].rolling(7).mean()                # 7-period moving average
ts['salary'].rolling('30D').mean()            # time-based window
ts['salary'].rolling(7, min_periods=1).mean()
ts['salary'].expanding().mean()              # cumulative
ts['salary'].ewm(span=10).mean()             # exponentially weighted

ts['salary'].shift(1)                        # lag
ts['salary'].diff()                         # first difference
ts['salary'].pct_change()
ts['salary'].shift(-1)                      # lead
```

INTERVIEW - resample vs groupby: `resample` is a `groupby` over time bins, requiring a `DatetimeIndex` (or `on=`), and it can produce bins that have no data (giving `NaN` rather than omitting them). `groupby(pd.Grouper(...))` is the general form.

INTERVIEW - "Compute a 7-day moving average of daily sales."

```
daily = df.set_index('ts').resample('D')['amount'].sum()
ma7    = daily.rolling(7, min_periods=1).mean()
```

Mention `min_periods` - without it the first six values are `NaN`, which is sometimes correct and sometimes a bug.

28. MultiIndex

```
mi = df.set_index(['dept', 'name']).sort_index()      # ALWAYS sort a MultiIndex
mi.loc['eng']                                       # all rows in dept 'eng'
mi.loc[('eng', 'Ana')]                             # a specific pair
mi.loc[(slice(None), 'Ana'), :]                    # cross-section on the second level
mi.xs('Ana', level='name')                          # cleaner cross-section
mi.index.get_level_values('dept')
mi.reset_index()
mi.swaplevel().sort_index()
mi.groupby(level='dept').sum()
```

GOTCHA - selecting on an unsorted MultiIndex raises `UnsortedIndexError` or silently degrades to a slow path. Call `.sort_index()` right after creating one.

29. Performance and Idioms

```
pd.options.mode.copy_on_write = True      # opt into Pandas 3.0 semantics early
```

Ranked list of speedups

1. **Vectorize.** Replace `apply/iterrows` with column arithmetic.
2. **Never use `iterrows`.** It builds a Series per row. If you truly must iterate, `itertuples(index=False)` is roughly 10x faster, and `zip(df.a, df.b)` faster still.
3. **Right dtypes.** `category`, downcast ints, `string[pyarrow]`.
4. **Avoid growing a DataFrame in a loop.** Append dicts to a list, then build once: `pd.DataFrame(rows)`.
5. **Use `merge` on indexed/sorted keys**, and `validate=` to catch fan-out.
6. **Use `query/eval`** on large frames to avoid temporaries.
7. **Use `pd.concat(list_of_frames)` once**, not repeated `pd.concat` in a loop.
8. **Consider Polars or DuckDB** when a single machine is the constraint and the operation is a scan-heavy aggregate.

```
# The anti-pattern
for i, row in df.iterrows():
    df.at[i, 'z'] = row.x * row.y      # slow, and mutating while iterating
# The fix
df['z'] = df.x * df.y
```

Method chaining style

```
result = (
    pd.read_csv('sales.csv', parse_dates=['date'])
    .rename(columns=str.lower)
    .query('amount > 0')
    .assign(month=lambda d: d.date.dt.to_period('M'),
            net=lambda d: d.amount - d.discount)
    .groupby(['month', 'region'], as_index=False)
    .agg(revenue=('net', 'sum'), orders=('id', 'size'))
    .sort_values('revenue', ascending=False)
    .reset_index(drop=True)
)
```

Chaining avoids intermediate variables and `SettingWithCopyWarning` entirely. Showing this style in an interview reads as senior.

30. Pandas Gotchas Cheat Sheet

Trap	Cause	Fix
<code>SettingWithCopyWarning</code>	chained indexing	single <code>.loc[rows, cols] = v</code>
int column became float	NaN forces upcast	nullable <code>Int64</code> dtype
<code>df[a > 1 & b < 2]</code>	<code>&</code> precedence	parenthesise each condition
merge row explosion	duplicate keys on right	<code>validate=</code> , dedupe first
<code>.str.contains</code> mask error	NaN in the mask	<code>na=False</code>
<code>inplace=True</code> returns None	by design	assign the result instead
<code>df.append</code> gone	removed in Pandas 2.0	<code>pd.concat</code>
MultIndex slicing errors	unsorted index	<code>.sort_index()</code>
<code>groupby</code> drops NaN keys	default <code>dropna=True</code>	<code>dropna=False</code>
std differs from NumPy	<code>ddof=1</code> vs <code>ddof=0</code>	pass <code>ddof</code> explicitly
dates parsed wrong	ambiguous D/M order	pass explicit <code>format=</code>
<code>apply</code> returns odd shape	pandas infers the result type	use <code>agg/transform</code>

Also: `inplace=True` does not generally save memory (Pandas usually copies internally anyway) and it breaks method chaining. The community consensus is to avoid it.

31. Pandas Interview Drill

Q1. Series vs DataFrame vs Index? Series is 1-D labelled data; DataFrame is a 2-D labelled table sharing a row Index across column Series; Index is the immutable label array enabling alignment and fast lookups.

Q2. loc vs iloc vs at vs iat? Label vs position; `at/iat` are single-scalar fast paths. `.loc` slices are end-inclusive.

Q3. How do you handle missing data? Detect (`isna().sum()`), decide per column based on null rate and mechanism (MCAR/MAR/MNAR), then drop, fill with a statistic, forward-fill for time series, or group-wise impute. Fit imputers on training data only.

Q4. apply vs map vs applymap? `map` is elementwise on a Series (and accepts a dict); `apply` works along an axis of a DataFrame or elementwise on a Series; `applymap` (now `DataFrame.map`) is elementwise on a DataFrame. All are slower than vectorized operations.

Q5. merge vs join vs concat? `merge` is the general SQL-style key join; `join` is a convenience wrapper that defaults to joining on the index; `concat` stacks objects along an axis with index alignment, not key matching.

Q6. How do you find and remove duplicates? `df.duplicated(subset=..., keep=...)` to inspect, `df.drop_duplicates(subset=..., keep='first')` to remove. Decide `keep` based on whether the latest or earliest record is authoritative - `sort_values('ts').drop_duplicates('id', keep='last')` is the common idiom.

Q7. Give the pandas equivalents of SQL.

SQL	Pandas
SELECT a, b	df[['a', 'b']]
WHERE a > 5	df[df.a > 5] or df.query('a > 5')
GROUP BY a	df.groupby('a')
HAVING count(*) > 2	.filter(lambda g: len(g) > 2)
ORDER BY a DESC	df.sort_values('a', ascending=False)
LIMIT 10	df.head(10)
JOIN	pd.merge(...)
UNION ALL	pd.concat(...)
DISTINCT	df.drop_duplicates()
RANK() OVER (PARTITION BY d)	df.groupby('d').x.rank()
SUM(x) OVER (PARTITION BY d)	df.groupby('d').x.transform('sum')
LAG(x)	df.x.shift(1)
CASE WHEN	np.select / np.where

Q8. How do you compute a rank within groups? `df.groupby('dept')`

`['salary'].rank(method='dense', ascending=False)`. Know the `method` options: `average` (default), `min`, `max`, `dense`, `first`.

Q9. Find the second-highest salary per department.

```
(df.sort_values('salary', ascending=False)
 .groupby('dept')
 .nth(1))
```

or `df[df.groupby('dept').salary.rank(method='dense', ascending=False) == 2]`.

Q10. How would you detect outliers? IQR rule ($q1 - 1.5 \cdot iqr$, $q3 + 1.5 \cdot iqr$), z-score ($(x - x.mean()) / x.std()$ beyond 3), or robust modified z-score using the median and MAD. State that the right method depends on distribution shape, and that "outlier" should be defined by domain, not by a formula.

Q11. What is the difference between `size()` and `count()` on a `groupby`? `size()` counts rows including NaN and returns a Series; `count()` counts non-null values per column and returns a DataFrame.

Q12. How is Pandas different from Spark/Polars/Dask? Pandas is single-machine, eager, and in-memory. Polars is single-machine but multi-threaded with a lazy query optimizer and Arrow memory. Dask parallelizes a pandas-like API across cores/machines. Spark is distributed and JVM-based with lazy DAG execution. Reach for the others when data exceeds RAM or when you need cluster parallelism.

Q13. What happens when you set an index? The chosen column becomes the row labels, enabling fast label lookups (hash-based for unique indexes, binary search for sorted ones) and automatic alignment in joins and arithmetic.

Q14. How do you concatenate frames with different columns? `pd.concat` takes the union of columns and fills missing entries with NaN. Pass `join='inner'` to keep only shared columns, and `ignore_index=True` when the original index is meaningless.

Q15. Explain Copy-on-Write. From Pandas 3.0, operations that returned ambiguous views now behave as if they returned copies, with lazy deferral so performance does not regress. Chained assignment silently stops working, so all mutation must go through a single `.loc/.iloc` call on the original object.

Coding exercises

1. Read a CSV and report the null rate, dtype, and cardinality of every column.

2. Given orders and customers, list customers with no orders (anti-join).
3. Compute month-over-month revenue growth by region.
4. For each user, find the time gap between consecutive events.
5. Flag the top 5% of transactions per merchant by amount.
6. Convert a wide quarterly report into long form and back.
7. Given duplicate records with timestamps, keep the latest per key.
8. Compute a 7-day rolling average that ignores missing days correctly.
9. Split a `full_name` column into first/last handling middle names.
10. Downsize a 2 GB DataFrame to under 500 MB without losing information.

PART III - Flask

32. The Mental Model

Flask is a **WSGI micro-framework**. Two words, both load-bearing.

WSGI (Web Server Gateway Interface, PEP 3333) is the contract between a Python web application and a web server. The application is any callable that accepts `(environ, start_response)` and returns an iterable of bytes. Gunicorn, uWSGI, and Waitress speak WSGI; so does Flask. This is why you never run Flask's built-in server in production - it is single-threaded by default, has no process management, and is explicitly documented as a development tool.

Micro means Flask ships routing, request/response handling, templating (via Jinja2), and a development server - and nothing else. No ORM, no form validation, no authentication, no admin. You assemble those from extensions. This is a design philosophy, not a limitation: Flask makes very few decisions for you, which is a strength on small services and a burden on large ones.

Flask stands on two Pallets libraries:

- **Werkzeug** - WSGI utilities: the request/response objects, the URL routing map, the development server, the debugger.
- **Jinja2** - the template engine.

```
from flask import Flask
app = Flask(__name__)

@app.route('/')
def index():
    return 'Hello, World!'

if __name__ == '__main__':
    app.run(debug=True)
```

```
flask --app app run --debug      # the modern CLI form
```

INTERVIEW - "What is WSGI and why does it matter?" It decouples the application from the server, so any WSGI app runs behind any WSGI server. It is synchronous and one-request-per-worker-thread by design, which is why async-heavy workloads led to ASGI (Starlette, FastAPI). Flask 2.0+ supports `async def` view functions, but it still runs them in a worker thread via `asgiref`, so you do not get true ASGI concurrency without deploying under an ASGI bridge.

33. Routing

```
@app.route('/about')
def about(): ...

@app.route('/user/<username>')           # string (default), no slashes
def user(username): ...

@app.route('/post/<int:post_id>')        # converters: int, float, path,
def post(post_id): ...                  # uuid, string
@app.route('/files/<path:subpath>')      # path accepts slashes

@app.route('/items', methods=['GET', 'POST'])
def items(): ...

@app.get('/items')                      # shorthand decorators
@app.post('/items')

@app.route('/items/', defaults={'page': 1})
@app.route('/items/page/<int:page>')
def items_page(page): ...
```

Trailing slashes

A rule ending in `/` (`/items/`) is treated as a directory: requesting `/items` issues a 308 redirect to `/items/`. A rule without the slash (`/items`) is a file: requesting `/items/` returns 404. Pick one convention and keep it; API clients that do not follow redirects will break otherwise.

URL building

```
from flask import url_for
url_for('user', username='ana')          # -> /user/ana
url_for('static', filename='app.css')    # -> /static/app.css
url_for('user', username='ana', _external=True) # absolute URL
url_for('.local_view')                   # relative to current blueprint
```

INTERVIEW - "Why `url_for` instead of hardcoding paths?" It survives route changes, handles URL quoting and converters correctly, respects the application root and blueprint prefixes, and works with `_external` for absolute links in emails.

34. The Request Object

```
from flask import request

request.method                # 'GET', 'POST', ...
request.args.get('q', '')    # query string ?q=...
request.args.getlist('tag')  # repeated params
request.form['email']         # form body (raises 400 if missing)
request.form.get('email')     # returns None if missing
request.get_json()            # parsed JSON body; raises 415 if content-type wrong
request.get_json(silent=True) # returns None instead of raising
request.files['avatar']       # uploaded file (FileStorage)
request.headers.get('Authorization')
request.cookies.get('session_id')
request.path; request.url; request.base_url
request.remote_addr
request.view_args             # the dict of URL converter values
```

`request.args` and `request.form` are `MultiDict` objects - dict-like but able to hold multiple values per key.

File uploads

```
import os
from werkzeug.utils import secure_filename

ALLOWED = {'png', 'jpg', 'pdf'}

@app.post('/upload')
def upload():
    f = request.files.get('file')
    if not f or f.filename == '':
        return {'error': 'no file'}, 400
    ext = f.filename.rsplit('.', 1)[-1].lower()
    if ext not in ALLOWED:
        return {'error': 'bad type'}, 400
    fname = secure_filename(f.filename) # strips ../ and unsafe chars
    f.save(os.path.join(app.config['UPLOAD_FOLDER'], fname))
    return {'saved': fname}, 201
```

INTERVIEW - always mention `secure_filename` (path traversal defence), `MAX_CONTENT_LENGTH` (denial-of-service defence), extension *and* content-type validation, and storing uploads outside the web root or in object storage.

35. Responses

```
return 'text' # 200, text/html
return 'not found', 404
return {'a': 1} # dict -> JSON automatically
return jsonify(a=1), 200, {'X-Custom': 'v'}
return make_response(render_template('p.html'), 200)
return redirect(url_for('index'), code=302)
return send_file('report.pdf', as_attachment=True)
return Response(generate(), mimetype='text/csv') # streaming
return '', 204 # no content
```

```

resp = make_response({'ok': True})
resp.set_cookie('token', 'abc', httponly=True, secure=True, samesite='Lax',
                max_age=3600)
resp.headers['Cache-Control'] = 'no-store'
return resp

```

Status codes worth knowing cold

Code	Meaning	Typical use
200	OK	successful GET/PUT
201	Created	successful POST, include Location
204	No Content	successful DELETE
400	Bad Request	malformed input
401	Unauthorized	missing/invalid credentials
403	Forbidden	authenticated but not permitted
404	Not Found	resource missing
409	Conflict	duplicate/version clash
422	Unprocessable	validation failure
429	Too Many Requests	rate limited
500	Server Error	unhandled exception

INTERVIEW - 401 vs 403: 401 means "*I don't know who you are*" (authentication); 403 means "*I know who you are, and you may not do this*" (authorization).

36. Templates (Jinja2)

```

from flask import render_template
@app.route('/hello/<name>')
def hello(name):
    return render_template('hello.html', name=name, items=[1,2,3])

```

```

<!-- templates/base.html -->
<!doctype html>
<html><body>
  <nav>{% block nav %}{% endblock %}</nav>
  <main>{% block content %}{% endblock %}</main>
</body></html>

<!-- templates/hello.html -->
{% extends "base.html" %}
{% block content %}
  <h1>Hello {{ name|title }}</h1>
  {% if items %}
    <ul>{% for i in items %}<li>{{ loop.index }}: {{ i }}</li>{% endfor %}</ul>
  {% else %}
    <p>No items.</p>
  {% endif %}
  {{ raw_html|safe }}
  {% include "footer.html" %}
{% endblock %}

```

Jinja2 **autoescapes** HTML in `.html` templates, which is your primary XSS defence. `|safe` disables it - only use it on content you generated, never on user input.

```

@app.template_filter('currency')
def currency(v): return f'${v:,.2f}'

@app.context_processor
def inject_globals(): return {'site_name': 'MyApp'}

```

INTERVIEW - "How does Flask prevent XSS?" Jinja2 autoescaping by default, plus `markupsafe`. Add a Content-Security-Policy header, and never render user data with `|safe` or build HTML by string concatenation.

37. Application and Request Contexts

This is the topic that separates people who have read the docs from people who have not.

Flask uses **context-local proxies**: `request`, `session`, `g`, and `current_app` are global-looking objects that actually resolve to whatever is bound to the current context (implemented with `contextvars`, so they are correct per thread *and* per async task).

- **Application context** - holds `current_app` and `g`. Pushed when handling a request, and also when you run a CLI command.
- **Request context** - holds `request` and `session`. Pushed for each incoming request, popped when the response is sent.

```

from flask import g, current_app

@app.before_request
def load_user():
    g.user = get_user_from_token(request.headers.get('Authorization'))

@app.teardown_appcontext
def close_db(exc):
    db = g.pop('db', None)
    if db is not None:
        db.close()

```

GOTCHA - `g` is **per request**, not per user and not global across requests. Storing anything there expecting it to persist is a bug.

Outside a request (a script, a test, a background job) you must push a context yourself:

```

with app.app_context():
    db.create_all()

with app.test_request_context('/items?q=x'):
    print(request.args['q'])

```

The classic error "*Working outside of application context*" means exactly this.

INTERVIEW - "**Why are `request` and `current_app` globals safe under concurrency?**" They are not real globals - they are proxies that look up the value bound to the current context variable. Each request (thread, greenlet, or task) gets its own binding, so there is no cross-talk.

38. Configuration

```

app.config['DEBUG'] = True
app.config.from_object('config.ProductionConfig')
app.config.from_pyfile('config.py')
app.config.from_envvar('APP_SETTINGS')
app.config.from_prefixed_env()           # reads FLASK_* environment variables

```

```

# config.py
import os
class Config:
    SECRET_KEY = os.environ['SECRET_KEY']           # fail loudly if unset
    SQLALCHEMY_DATABASE_URI = os.environ['DATABASE_URL']
    JSON_SORT_KEYS = False

class Dev(Config):
    DEBUG = True
    SQLALCHEMY_DATABASE_URI = 'sqlite:///dev.db'

class Prod(Config):
    DEBUG = False
    SESSION_COOKIE_SECURE = True
    SESSION_COOKIE_HTTPONLY = True
    SESSION_COOKIE_SAMESITE = 'Lax'

```

Never commit secrets. Use environment variables, a `.env` file excluded from git (loaded with `python-dotenv`), or a secrets manager. `SECRET_KEY` signs the session cookie; if it leaks, an attacker can forge sessions.

INTERVIEW - "Is it safe to run with `debug=True` in production?" No. The Werkzeug debugger exposes an interactive Python console on tracebacks. With the PIN bypassed or guessed it is remote code execution. Debug mode also disables some error handling and leaks source code.

39. The Application Factory and Blueprints

A single-file `app = Flask(__name__)` at import time causes circular imports and makes testing awkward, because the app is configured once at module load. The **application factory** fixes both.

```
# app/__init__.py
from flask import Flask
from .extensions import db, migrate

def create_app(config_object='config.Dev'):
    app = Flask(__name__)
    app.config.from_object(config_object)

    db.init_app(app)                # bind extensions to THIS app
    migrate.init_app(app, db)

    from .api.users import bp as users_bp
    from .api.items import bp as items_bp
    app.register_blueprint(users_bp, url_prefix='/api/users')
    app.register_blueprint(items_bp, url_prefix='/api/items')

    register_error_handlers(app)
    return app
```

```
# app/extensions.py -- extensions created WITHOUT an app
from flask_sqlalchemy import SQLAlchemy
from flask_migrate import Migrate
db = SQLAlchemy()
migrate = Migrate()
```

```
# app/api/users.py
from flask import Blueprint, request
bp = Blueprint('users', __name__)

@bp.get('/')
def list_users():
    return {'users': []}

@bp.errorhandler(404)
def nf(e): return {'error': 'not found'}, 404
```

INTERVIEW - "What is a Blueprint?" A reusable, registerable group of routes, templates, static files, and handlers. It enables modular structure, URL prefixes, per-module error handlers and hooks, and registering the same blueprint more than once under different prefixes. It is Flask's answer to Django apps.

INTERVIEW - "Why the application factory pattern?" Deferred, parameterized construction: you can build one app with test config and another with production config in the same process, avoid circular

imports, and support multiple app instances. Extensions are instantiated at module level and bound with `init_app`.

Recommended project layout

```
myapp/
  app/
    __init__.py      # create_app()
    extensions.py    # db, migrate, jwt, cors ...
    models/         # SQLAlchemy models
    api/            # blueprints
    services/       # business logic, NOT in the views
    schemas/        # marshmallow / pydantic validation
    errors.py
  migrations/
  tests/
  config.py
  wsgi.py           # app = create_app()
  requirements.txt
  Dockerfile
```

40. Databases with Flask-SQLAlchemy

```
# models.py
from .extensions import db
from datetime import datetime

class User(db.Model):
    __tablename__ = 'users'
    id = db.Column(db.Integer, primary_key=True)
    email = db.Column(db.String(255), unique=True, nullable=False, index=True)
    name = db.Column(db.String(120), nullable=False)
    created = db.Column(db.DateTime, default=datetime.utcnow)
    items = db.relationship('Item', back_populates='owner',
                           cascade='all, delete-orphan', lazy='selectin')

    def to_dict(self):
        return {'id': self.id, 'email': self.email, 'name': self.name}

class Item(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    title = db.Column(db.String(200), nullable=False)
    user_id = db.Column(db.Integer, db.ForeignKey('users.id'), nullable=False)
    owner = db.relationship('User', back_populates='items')
```

```
# queries
db.session.add(user); db.session.commit()
User.query.get(1) # legacy style
db.session.get(User, 1) # SQLAlchemy 2.0 style
User.query.filter_by(email='a@b.c').first()
User.query.filter(User.name.ilike('%ana%')).all()
User.query.order_by(User.created.desc()).paginate(page=1, per_page=20)
db.session.execute(db.select(User).where(User.id == 1)).scalar_one_or_none()
db.session.delete(user); db.session.commit()
```

Migrations

```
flask db init
flask db migrate -m "add users table"
flask db upgrade
flask db downgrade
```

INTERVIEW - the N+1 query problem. Iterating `users` and touching `user.items` inside the loop issues one query per user. Fix with eager loading: `lazy='selectin'` on the relationship, or `User.query.options(selectinload(User.items))` per query. Being able to name and fix N+1 is one of the most reliable signals of backend experience.

Session hygiene: one session per request; commit once at the end; roll back on exception; use `db.session.rollback()` in your error handler; never share a session across threads.

41. Building a REST API

```
from flask import Blueprint, request, jsonify, abort
from .models import Item
from .extensions import db

bp = Blueprint('items', __name__)

@bp.get('/')
def list_items():
    page = request.args.get('page', 1, type=int)
    per = min(request.args.get('per_page', 20, type=int), 100)
    q = Item.query
    if s := request.args.get('search'):
        q = q.filter(Item.title.ilike(f'%{s}%'))
    p = q.order_by(Item.id).paginate(page=page, per_page=per, error_out=False)
    return {
        'data': [i.to_dict() for i in p.items],
        'meta': {'page': p.page, 'pages': p.pages, 'total': p.total},
    }

@bp.get('/<int:item_id>')
def get_item(item_id):
    item = db.session.get(Item, item_id) or abort(404)
    return item.to_dict()

@bp.post('/')
def create_item():
    data = request.get_json(silent=True) or {}
    if not data.get('title'):
        return {'error': 'title is required'}, 422
    item = Item(title=data['title'], user_id=data['user_id'])
    db.session.add(item)
    db.session.commit()
    return item.to_dict(), 201, {'Location': f'/api/items/{item.id}'}

@bp.put('/<int:item_id>')
def update_item(item_id):
    item = db.session.get(Item, item_id) or abort(404)
    data = request.get_json() or {}
    item.title = data.get('title', item.title)
    db.session.commit()
    return item.to_dict()

@bp.delete('/<int:item_id>')
def delete_item(item_id):
    item = db.session.get(Item, item_id) or abort(404)
    db.session.delete(item)
    db.session.commit()
    return '', 204
```

REST design principles worth reciting

- Resources are **nouns**, not verbs: `/items/5`, not `/getItem?id=5`.
- HTTP verbs carry the action; status codes carry the outcome.
- **Idempotency**: GET, PUT, DELETE are idempotent; POST is not. PATCH may or may not be.
- **Statelessness**: each request carries everything needed to serve it. No server-side per-client state between requests (this is what makes horizontal scaling possible).
- Version the API: `/api/v1/...`.

- Paginate every collection endpoint. Offset pagination is simple; cursor (keyset) pagination is correct under concurrent writes.
- Return consistent error envelopes.

Validation with Marshmallow

```
from marshmallow import Schema, fields, validate, ValidationError

class ItemSchema(Schema):
    id      = fields.Int(dump_only=True)
    title   = fields.Str(required=True, validate=validate.Length(min=1, max=200))
    user_id = fields.Int(required=True)

schema = ItemSchema()

@bp.post('/')
def create():
    try:
        data = schema.load(request.get_json() or {})
    except ValidationError as err:
        return {'errors': err.messages}, 422
    ...
```

Never trust client input. Validate types, ranges, and lengths at the boundary; keep the view thin and push business rules into a service layer.

42. Sessions, Cookies, and Authentication

```
from flask import session
session['user_id'] = 42          # signed cookie, client-side storage
session.get('user_id')
session.pop('user_id', None)
session.permanent = True
app.permanent_session_lifetime = timedelta(days=7)
```

INTERVIEW - "Where does Flask store the session?" By default in a cookie on the client, serialized and **cryptographically signed** with `SECRET_KEY`. Signed means tamper-evident, **not encrypted** - the user can read the contents. Never put secrets in it. For server-side sessions use Flask-Session with Redis.

JWT authentication

```
from flask_jwt_extended import (JWTManager, create_access_token,
                                jwt_required, get_jwt_identity)

jwt = JWTManager(app)

@app.post('/login')
def login():
    data = request.get_json()
    user = User.query.filter_by(email=data['email']).first()
    if not user or not check_password_hash(user.pw_hash, data['password']):
        return {'error': 'bad credentials'}, 401
    return {'access_token': create_access_token(identity=str(user.id))}

@app.get('/me')
@jwt_required()
def me():
    return {'id': get_jwt_identity()}
```

INTERVIEW - sessions vs JWT.

	Server session	JWT
State	server-side store	stateless, self-contained
Revocation	easy (delete the record)	hard (needs a denylist)
Scaling	needs shared store	no shared store needed
Size	small cookie id	larger token on every request
Best for	classic web apps	APIs, mobile, cross-service

Say that JWTs should be short-lived with a refresh token, must have their algorithm pinned (reject `alg: none`), and should be stored in an `HttpOnly` cookie rather than `localStorage` when XSS is a concern.

Password handling

```
from werkzeug.security import generate_password_hash, check_password_hash
hash_ = generate_password_hash(pw)           # PBKDF2/scrypt with a random salt
check_password_hash(hash_, candidate)
```

Never store plaintext, never use MD5/SHA1 alone, always salt, prefer a slow KDF (bcrypt, scrypt, argon2).

Decorator-based authorization

```
from functools import wraps

def role_required(role):
    def deco(fn):
        @wraps(fn)
        def wrapper(*a, **kw):
            if not g.user or g.user.role != role:
                abort(403)
            return fn(*a, **kw)
        return wrapper
    return deco

@bp.delete('/<int:i>')
@role_required('admin')
def delete(i): ...
```

`@wraps` matters: without it, two decorated views share the name `wrapper` and Flask raises a duplicate-endpoint error. That is a favourite interview gotcha.

43. Error Handling and Logging

```
from werkzeug.exceptions import HTTPException

@app.errorhandler(404)
def not_found(e): return {'error': 'not found'}, 404

@app.errorhandler(HTTPException)
def http_err(e):
    return {'error': e.name, 'detail': e.description}, e.code

@app.errorhandler(Exception)
def unhandled(e):
    db.session.rollback()
    app.logger.exception('unhandled error')    # logs the traceback
    return {'error': 'internal server error'}, 500

class APIError(Exception):
    def __init__(self, message, status=400):
        super().__init__(message); self.message, self.status = message, status

@app.errorhandler(APIError)
def api_err(e): return {'error': e.message}, e.status
```

```
import logging
from logging.handlers import RotatingFileHandler
h = RotatingFileHandler('app.log', maxBytes=10_000_000, backupCount=5)
h.setFormatter(logging.Formatter(
    '%(asctime)s %(levelname)s %(name)s %(message)s'))
app.logger.addHandler(h)
app.logger.setLevel(logging.INFO)
```

In production, log JSON to stdout and let the platform collect it; attach a request id to every log line for tracing; never log credentials, tokens, or personal data.

44. Hooks and Middleware

```
@app.before_request
def before(): g.start = time.time()           # returning a response SHORT-CIRCUITS

@app.after_request
def after(resp):                               # skipped if an exception escaped
    resp.headers['X-Response-Time'] = f'{time.time() - g.start:.3f}'
    return resp                                # MUST return the response

@app.teardown_request
def teardown(exc): ...                        # ALWAYS runs, even on exception

@app.teardown_appcontext
def teardown_app(exc): ...
```

INTERVIEW - after_request vs teardown_request: `after_request` receives and must return the response, and is skipped when an unhandled exception occurs; `teardown_request` always runs, receives the exception (or None), and cannot modify the response. Cleanup belongs in `teardown`.

True WSGI middleware

```
class TimingMiddleware:
    def __init__(self, app): self.app = app
    def __call__(self, environ, start_response):
        # runs BELOW Flask, sees the raw WSGI protocol
        return self.app(environ, start_response)

app.wsgi_app = TimingMiddleware(app.wsgi_app)

from werkzeug.middleware.proxy_fix import ProxyFix
app.wsgi_app = ProxyFix(app.wsgi_app, x_for=1, x_proto=1)  # behind nginx
```

`ProxyFix` is required behind a reverse proxy so `request.remote_addr` and generated URLs reflect the real client and scheme instead of the proxy.

45. CORS, Rate Limiting, Security

```
from flask_cors import CORS
CORS(app, resources={r"/api/*": {"origins": ["https://myfrontend.com"]}},
    supports_credentials=True)

from flask_limiter import Limiter
from flask_limiter.util import get_remote_address
limiter = Limiter(get_remote_address, app=app, default_limits=["200/hour"],
    storage_uri="redis://localhost:6379")

@bp.post('/login')
@limiter.limit("5/minute")
def login(): ...
```

INTERVIEW - "What is CORS and why do you hit it?" The browser's same-origin policy blocks cross-origin reads by default. CORS is the server telling the browser, via `Access-Control-Allow-*` headers, which origins may read the response. Non-simple requests trigger a preflight `OPTIONS`. Note this is a

browser restriction - curl and server-to-server calls are unaffected. Never set `Access-Control-Allow-Origin: *` together with credentials.

Security checklist

- SQL injection: use the ORM or parameterized queries; never f-string SQL.
- XSS: keep Jinja autoescaping on; add a CSP header.
- CSRF: use Flask-WTF's CSRF tokens for cookie-authenticated form posts; `SameSite` cookies help but are not a complete defence.
- Secrets in environment variables, never in git.
- `SESSION_COOKIE_SECURE`, `HTTPONLY`, `SAMESITE`; HTTPS everywhere with HSTS.
- `MAX_CONTENT_LENGTH` to cap request bodies.
- Rate-limit authentication endpoints.
- Pin dependencies and scan them (`pip-audit`).
- Never run with `debug=True`.

46. Testing

```
# conftest.py
import pytest
from app import create_app
from app.extensions import db as _db

@pytest.fixture
def app():
    app = create_app('config.Test')      # SQLALCHEMY_DATABASE_URI=sqlite:///memory:
    with app.app_context():
        _db.create_all()
        yield app
        _db.drop_all()

@pytest.fixture
def client(app):
    return app.test_client()

@pytest.fixture
def runner(app):
    return app.test_cli_runner()
```

```
def test_list_items_empty(client):
    r = client.get('/api/items/')
    assert r.status_code == 200
    assert r.get_json()['data'] == []

def test_create_item(client):
    r = client.post('/api/items/', json={'title': 'x', 'user_id': 1})
    assert r.status_code == 201
    assert r.get_json()['title'] == 'x'

def test_create_item_validation(client):
    r = client.post('/api/items/', json={})
    assert r.status_code == 422

def test_auth_required(client):
    assert client.get('/me').status_code == 401
```

The test client drives the WSGI app directly - no network, no running server, very fast. **INTERVIEW:** mention `app.test_client()`, fixtures for app/db isolation, a separate test config with an in-memory database, transaction rollback between tests, and mocking external HTTP calls (`responses`, `unittest.mock`).

47. Async, Background Jobs, and Concurrency

```
@app.get('/slow')
async def slow():
    await asyncio.sleep(1)
    return {'ok': True}
```

Flask runs `async def` views in an event loop inside a worker thread. It is convenient for calling async libraries, but it does **not** give you ASGI-level concurrency: the WSGI worker is still occupied for the duration.

Long work does not belong in a request at all. Use a queue:

```
from celery import Celery
celery = Celery(__name__, broker='redis://localhost:6379/0')

@celery.task
def send_report(user_id): ...

@bp.post('/report')
def report():
    task = send_report.delay(user_id)
    return {'task_id': task.id}, 202  # 202 Accepted
```

INTERVIEW - "A request takes 30 seconds. What do you do?" Do not block the worker. Enqueue the job (Celery/RQ/Dramatiq with Redis or RabbitMQ), return 202 with a task id, and let the client poll a status endpoint or receive a webhook. Mention idempotency keys and retries with backoff.

Concurrency model

Gunicorn runs *N* worker *processes*; each worker handles one request at a time with the sync worker class. Rules of thumb: `workers = 2 * cores + 1` for CPU-bound, and gthread/gevent workers for I/O-bound. Because workers are separate processes, **module-level global state is not shared between them** - a common source of "my in-memory cache works locally but not in production".

48. Deployment

```
gunicorn -w 4 -k gthread --threads 4 -b 0.0.0.0:8000 \
    --timeout 60 --access-logfile - 'wsgi:app'
```

```
# wsgi.py
from app import create_app
app = create_app('config.Prod')
```

```
FROM python:3.12-slim
ENV PYTHONDONTWRITEBYTECODE=1 PYTHONUNBUFFERED=1
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
EXPOSE 8000
CMD ["gunicorn", "-w", "4", "-b", "0.0.0.0:8000", "wsgi:app"]
```

```
server {
    listen 80;
    server_name api.example.com;
    location / {
        proxy_pass http://127.0.0.1:8000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
    location /static/ { alias /app/static/; expires 30d; }
}
```

INTERVIEW - "Why nginx in front of gunicorn?" TLS termination, static file serving, buffering slow clients (protecting workers from slowloris-style starvation), compression, request limits, and load balancing across app servers.

Production checklist

1. `debug=False`, `SECRET_KEY` from the environment.
2. Gunicorn/uWSGI with a sensible worker count and timeout.
3. `ProxyFix` behind a reverse proxy.
4. HTTPS + HSTS, secure cookie flags.
5. Migrations run as a deploy step, never automatically at import.
6. Health endpoint (`/healthz`) and readiness checks.
7. Structured logs to stdout, error tracking (Sentry).
8. Metrics: latency percentiles, error rate, saturation.
9. Connection pool sized against the database's max connections.
10. Graceful shutdown and zero-downtime deploys.

49. Flask vs Django vs FastAPI

	Flask	Django	FastAPI
Philosophy	micro, assemble it	batteries included	modern, typed
Protocol	WSGI (sync)	WSGI/ASGI	ASGI (async)
ORM	bring your own	built-in	bring your own
Admin	none	built-in	none
Validation	extension	forms	Pydantic, built-in
Docs	extension	extension	automatic OpenAPI
Best for	small/medium services, flexibility	large CRUD apps, fast start	high-concurrency typed APIs

INTERVIEW answer: choose Django when you need auth, admin, and ORM on day one; Flask when you want control and a small surface; FastAPI when you need async I/O concurrency and automatic schema/docs. Avoid saying one is simply "better".

50. Flask Interview Drill

Q1. What is Flask and what is it built on? A WSGI micro-framework built on Werkzeug (WSGI/routing/debugging) and Jinja2 (templating).

Q2. What does the `@app.route` decorator actually do? Registers a URL rule in the app's `url_map` bound to an endpoint name (the function name by default); at request time Werkzeug matches the URL and dispatches to the view.

Q3. Application context vs request context? App context carries `current_app` and `g` (per request, or per CLI invocation); request context carries `request` and `session`. Both are pushed/popped around each request.

Q4. What is `g` for? Per-request scratch storage, typically a database connection or the authenticated user, shared across hooks and views within one request only.

Q5. Why is `current_app` a proxy? So that extensions and modules can import it at module load without an app existing yet, and so it resolves to the correct app instance in multi-app or factory setups.

Q6. How do you structure a large Flask app? Application factory + blueprints + an extensions module + service layer + schemas + config classes, with tests using a test config.

Q7. How does Flask handle concurrent requests? The WSGI server does: worker processes and/or threads. Flask itself is synchronous. Context-locals keep per-request data isolated.

Q8. What is `jsonify` and how does it differ from `json.dumps`? It serializes and returns a `Response` with `application/json` content type and Flask's JSON provider settings (including datetime handling). Returning a plain dict from a view does this automatically in Flask 2.2+.

Q9. How do you upload and validate files safely? `request.files`, `secure_filename`, extension and MIME allow-lists, `MAX_CONTENT_LENGTH`, store outside the web root, and generate new filenames rather than trusting the client's.

Q10. How do you protect against SQL injection? Use the ORM or parameterized queries (`text("... :id")`, `{"id": id}`). Never format user input into SQL.

Q11. What is CSRF and how does Flask handle it? An attacker causes an authenticated user's browser to submit a request. Defence: per-session unpredictable tokens on state-changing requests (Flask-WTF), plus `SameSite` cookies. Token-authenticated APIs that do not use cookies are largely immune.

Q12. What is the N+1 problem and how do you fix it? Lazy relationship loading issues one query per parent row. Fix with eager loading (`selectinload`, `joinedload`) or an explicit join.

Q13. How do you run a task on every request? `@app.before_request` / `@app.after_request`. Note that `before_first_request` was removed in Flask 2.3; do one-time setup inside the factory instead.

Q14. How do you test a Flask app? `app.test_client()` with pytest fixtures, a dedicated test config, an in-memory or per-test database, and mocks for outbound calls.

Q15. How do you version an API? URL prefix (`/api/v1`) is the common, cache-friendly choice; header-based versioning is cleaner in theory and harder in practice. Blueprints make URL versioning trivial.

Q16. How would you scale a Flask app? Stateless workers behind a load balancer, horizontal scaling of gunicorn processes/containers, Redis for sessions and caching, read replicas and connection pooling for the database, a CDN for static content, and background queues for slow work.

PART IV - Streamlit

51. The Mental Model

Streamlit turns a plain Python script into a web app. There is no callback graph to declare, no HTML, no JavaScript, no routing. You write top-to-bottom Python and Streamlit renders it.

The price of that simplicity is one rule you must internalize:

On every user interaction, Streamlit re-runs your entire script from top to bottom.

Not a fragment. Not a diff. The whole file, every time a widget changes. Once you accept that, everything else - caching, session state, forms, fragments - exists to manage the consequences.

```
import streamlit as st

st.title('My First App')
name = st.text_input('Your name')
if name:
    st.write(f'Hello, {name}!!')
```

```
streamlit run app.py
streamlit run app.py --server.port 8502 --server.headless true
```

What happens on a rerun

1. The browser sends the new widget value over a WebSocket.
2. The server re-executes the script in a fresh run, in a thread tied to that user's **session**.
3. Widgets are re-created. Each one looks up its stored value by key and returns it, so the UI appears to "remember".
4. Streamlit diffs the produced element tree against the previous one and sends only the changes to the browser.

Two consequences that trip up every beginner:

- **Local variables do not persist across reruns.** A counter defined as `count = 0` resets on every interaction. That is what `st.session_state` is for.
- **Expensive work re-executes on every interaction** unless it is cached.

INTERVIEW - "Explain Streamlit's execution model." Full top-to-bottom script rerun per interaction, per-session server-side state, WebSocket-driven element diffing. Then name the three tools that make it practical: `st.cache_data`, `st.cache_resource`, and `st.session_state`.

52. Display Elements

```
st.title('Title'); st.header('H'); st.subheader('Sub')
st.text('fixed width'); st.markdown('**bold** and `code`')
st.caption('small grey text')
st.code('print(1)', language='python')
st.latex(r'\sum_{i=1}^n x_i')
st.divider()
st.write(anything)           # the swiss-army display: str, df, fig, dict, model
'Just a string'              # "magic": a bare expression is displayed

st.dataframe(df, use_container_width=True, hide_index=True)
st.data_editor(df, num_rows='dynamic')    # EDITABLE grid, returns edited df
st.table(df)                             # static, no scrolling
st.metric('Revenue', '1.2M', delta='+4.5%')
st.json({'a': 1})

st.image('logo.png', width=200)
st.audio('a.mp3'); st.video('v.mp4')

st.success('done'); st.info('fyi'); st.warning('careful'); st.error('failed')
st.exception(err)
st.balloons(); st.toast('Saved!', icon='OK')
```

`st.write` dispatches on type: a DataFrame renders as an interactive table, a Matplotlib/Plotly/Altair figure as a chart, a dict as JSON, a function as its docstring.

53. Widgets

```
st.button('Click me')           # True only on the run it was clicked
st.checkbox('Agree')
st.toggle('Dark mode')
st.radio('Pick', ['a', 'b'], horizontal=True)
st.selectbox('Choose', options, index=0)
st.multiselect('Tags', options, default=[])
st.slider('Age', 0, 100, 25, step=1)
st.select_slider('Size', options=['S', 'M', 'L'])
st.number_input('Qty', min_value=0, max_value=10, value=1)
st.text_input('Name', placeholder='...')
st.text_area('Bio', height=120)
st.date_input('Date'); st.time_input('Time')
st.color_picker('Colour', '#00f900')
st.file_uploader('CSV', type=['csv'], accept_multiple_files=False)
st.camera_input('Take a photo')
st.download_button('Download', data=csv_bytes, file_name='out.csv',
                  mime='text/csv')
st.pills('Filter', ['A', 'B', 'C'], selection_mode='multi')
st.segmented_control('View', ['Table', 'Chart'])
st.feedback('stars')
st.link_button('Docs', 'https://docs.streamlit.io')
```

Every widget returns its current value on every rerun, and accepts:

```
st.slider('x', 0, 10, key='x_slider',          # stable id -> st.session_state.x_slider
        help='tooltip', disabled=False,
        on_change=callback, args=(), kwargs={},
        label_visibility='visible')          # 'hidden' | 'collapsed'
```

The button trap

```
if st.button('Load'):
    data = expensive()          # runs ONCE
if st.checkbox('Show'):        # any other interaction reruns the script,
    st.write(data)              # `data` no longer exists -> NameError
```

`st.button` returns `True` only during the single rerun triggered by the click. On the next rerun it is `False` again. **Never nest state-changing logic inside `if st.button(...)` without persisting the result to session state.**

```
if st.button('Load'):
    st.session_state.data = expensive()
if 'data' in st.session_state:
    st.write(st.session_state.data)
```

INTERVIEW - this exact scenario is the most common Streamlit debugging question. Being able to explain *why* it happens (buttons are momentary, script reruns are total) is the whole answer.

Widget keys and duplicate IDs

Streamlit identifies a widget by a hash of its type, label, and parameters. Two widgets with identical arguments raise `DuplicateWidgetID`. Fix with an explicit `key=`. Keys are also how you read or set a widget's value from `st.session_state`.

54. Session State

`st.session_state` is a per-user, per-browser-session dictionary that survives reruns (but not a page refresh, and not a server restart).

```
if 'count' not in st.session_state:
    st.session_state.count = 0          # initialize ONCE, guarded

st.session_state.count += 1
st.session_state['count']              # dict access works too
del st.session_state['count']
st.session_state.get('count', 0)
```

Widgets are bound to keys

```
st.text_input('Name', key='name')
st.write(st.session_state.name)        # same value, no return variable
```

GOTCHA - you may set `st.session_state.name` *before* the widget is created, but modifying it after the widget exists on the same run raises `StreamlitAPIException`. The safe pattern is a callback.

Callbacks

```
def reset():
    st.session_state.count = 0

st.button('Reset', on_click=reset)
st.number_input('N', key='n', on_change=lambda: st.write('changed'))
```

Callbacks run **before** the script reruns, which makes them the right place to mutate state that the script body will then read.

Common state patterns

```
# accumulate a list
st.session_state.setdefault('log', [])
if st.button('Add'):
    st.session_state.log.append(datetime.now())

# multi-step wizard
st.session_state.setdefault('step', 1)
if st.session_state.step == 1:
    ...
    if st.button('Next'): st.session_state.step = 2; st.rerun()

# chat history
st.session_state.setdefault('messages', [])
for m in st.session_state.messages:
    with st.chat_message(m['role']):
        st.markdown(m['content'])
if prompt := st.chat_input('Say something'):
    st.session_state.messages.append({'role': 'user', 'content': prompt})
    st.rerun()
```

INTERVIEW - "Is session state shared between users?" No. Each browser session gets its own isolated state on the server. It is not persistent storage: a refresh clears it. For persistence use a database, a file, cookies, or `st.query_params`.

55. Caching - `cache_data` VS `cache_resource`

Without caching, every interaction re-reads your CSV and re-queries your database. This is the single biggest performance lever in Streamlit.

```
@st.cache_data(ttl=600, max_entries=100, show_spinner='Loading...')
def load(path: str) -> pd.DataFrame:
    return pd.read_csv(path)

@st.cache_resource
def get_engine():
    return create_engine(os.environ['DATABASE_URL'])
```

	st.cache_data	st.cache_resource
For	data: DataFrames, arrays, dicts, API results	global objects: DB connections, ML models, clients
Returns	a new copy each call	the same object , shared
Serializable	must be picklable	need not be
Shared across sessions	yes (keyed by args), copies handed out	yes, the identical instance
Mutation risk	none - you get a copy	high - all users share it

INTERVIEW - "When would you use `cache_resource` over `cache_data`?" For objects that are expensive to create, must not be duplicated, and are not picklable - database engines, ML models, API clients. Add the warning: the object is shared across all sessions and threads, so it must be thread-safe, and mutating it affects every user.

How the cache key is computed

Streamlit hashes the function's source code plus the arguments. Unhashable arguments raise `UnhashableParamError`. Two fixes:

```
@st.cache_data
def q(_conn, sql: str):          # leading underscore -> EXCLUDED from the hash
    return pd.read_sql(sql, _conn)

@st.cache_data(hash_funcs={MyClass: lambda o: o.id})
def f(obj): ...
```

```
st.cache_data.clear()           # clear all data caches
load.clear()                    # clear one function's cache
```

GOTCHA - `cache_data` returns a *copy*, so mutating the returned DataFrame does not corrupt the cache. But the copy costs time and memory on large frames; if a function returns a 2 GB DataFrame, every call pays a 2 GB copy. Cache smaller, aggregated results where you can.

GOTCHA - caching a function whose result depends on something not in its arguments (the current time, a global, a file that changes) will serve stale results. Pass the dependency in explicitly or set a `ttl`.

56. Layout

```
c1, c2, c3 = st.columns(3)
c1.metric('Users', 120)
with c2:
    st.line_chart(df)
left, right = st.columns([2, 1], gap='large')    # relative widths

tab1, tab2 = st.tabs(['Data', 'Chart'])
with tab1: st.dataframe(df)

with st.sidebar:
    st.header('Filters')
    region = st.selectbox('Region', regions)
    st.sidebar.slider('Threshold', 0, 100)        # equivalent shorthand

with st.expander('Advanced options', expanded=False):
    st.checkbox('Verbose')

with st.popover('Settings'):
    st.radio('Theme', ['light', 'dark'])

placeholder = st.empty()                        # a single slot you can overwrite later
placeholder.write('loading...')
placeholder.dataframe(df)                      # replaces the previous content

with st.container(height=300, border=True):
    st.write('scrollable box')

@st.dialog('Confirm')
def confirm():
    st.write('Are you sure?')
    if st.button('Yes'): st.session_state.done = True; st.rerun()
```

Progress and status

```
with st.spinner('Crunching...'):
    result = heavy()

bar = st.progress(0, text='working')
for i in range(100):
    bar.progress(i + 1)

with st.status('Running pipeline', expanded=True) as s:
    st.write('extracting'); st.write('transforming')
    s.update(label='Done', state='complete')
```

57. Charts and Data Display

```
st.line_chart(df, x='date', y='sales', color='region')
st.bar_chart(df); st.area_chart(df); st.scatter_chart(df, x='a', y='b')
st.map(df)                                # needs 'lat'/'lon' columns

import plotly.express as px
st.plotly_chart(px.bar(df, x='dept', y='salary'), use_container_width=True)

import altair as alt
st.altair_chart(alt.Chart(df).mark_circle().encode(x='a', y='b'))

import matplotlib.pyplot as plt
fig, ax = plt.subplots(); ax.plot([1,2,3])
st.pyplot(fig)
```

The built-in `st.*_chart` functions are Altair wrappers - fast to write, limited to configure. Use Plotly or Altair directly when you need control, interaction, or custom tooltips.

Editable tables

```
edited = st.data_editor(
    df,
    num_rows='dynamic',
    disabled=['id'],
    column_config={
        'salary': st.column_config.NumberColumn('Salary', format='${d}',
                                                  min_value=0),
        'url': st.column_config.LinkColumn(),
        'active': st.column_config.CheckboxColumn(),
        'trend': st.column_config.LineChartColumn(),
    },
)
if st.button('Save'):
    write_back(edited)
```

58. Forms

Every widget change triggers a rerun. When a user must fill five fields before anything should happen, that is five wasted reruns. A form batches them.

```
with st.form('signup', clear_on_submit=True, border=True):
    name = st.text_input('Name')
    email = st.text_input('Email')
    age = st.number_input('Age', 0, 120)
    submitted = st.form_submit_button('Sign up')

if submitted:
    if not email or '@' not in email:
        st.error('Invalid email')
    else:
        save(name, email, age)
        st.success('Saved')
```

Rules: widgets inside a form do not trigger reruns until submit; a form must contain exactly one `st.form_submit_button`; `st.button` is not allowed inside a form; and widgets inside a form cannot have `on_change` callbacks.

INTERVIEW - "Why use `st.form`?" To batch multiple widget inputs into one rerun, which avoids partial-state processing and avoids repeating expensive work on every keystroke.

59. Multipage Apps and Navigation

The modern approach (Streamlit 1.36+) declares pages in code:

```
# streamlit_app.py
import streamlit as st

pages = {
    'Explore': [
        st.Page('pages/overview.py', title='Overview', icon=':material/home:'),
        st.Page('pages/detail.py', title='Detail'),
    ],
    'Admin': [
        st.Page(settings_fn, title='Settings', url_path='settings'),
    ],
}
pg = st.navigation(pages)
pg.run()
```

The legacy approach is a `pages/` directory next to your entrypoint, with files named `1_Overview.py`, `2_Detail.py` - the numeric prefix sets the order and is stripped from the display name.

```
st.switch_page('pages/detail.py')      # programmatic navigation
st.page_link('pages/detail.py', label='Go to detail')

st.query_params                        # read/write URL parameters
st.query_params['id'] = 42             # survives a refresh; good for sharing
```

INTERVIEW - "How do you share state between pages?" `st.session_state` persists across page switches within a session; `st.query_params` survives a refresh and makes URLs shareable; anything that must outlive the session goes in a database or file.

60. Fragments, Reruns, and Performance

```
@st.fragment(run_every='5s')
def live_panel():
    st.metric('Now', fetch_latest())    # reruns ALONE, not the whole script

live_panel()
```

Fragments are the answer to "my app has one auto-refreshing widget and everything else recomputes". Interactions with widgets *inside* a fragment rerun only that fragment.

```
st.rerun()                            # force an immediate rerun from the top
st.stop()                              # halt execution of the rest of the script
```

Performance checklist

1. **Cache everything expensive** - `cache_data` for data, `cache_resource` for connections and models.
2. **Filter at the source**, not in Python: push `WHERE` and `LIMIT` into SQL.
3. **Do not render enormous DataFrames**. Paginate or sample.
4. **Use forms** to avoid a rerun per keystroke.
5. **Use fragments** to isolate frequently-updating regions.
6. **Do not put heavy work at module top level** unguarded - it runs on every rerun for every session.
7. **Cache with `ttl`** for anything fetched from an API.
8. **Prefer aggregated results**: cache the 1,000-row summary, not the 10,000,000-row raw table.

61. Secrets, Connections, Config, and Theming

```
# .streamlit/secrets.toml -- NEVER commit this
DB_PASSWORD = "supersecret"

[connections.mydb]
type = "sql"
url = "postgresql://user:pass@host:5432/db"
```

```
st.secrets['DB_PASSWORD']

conn = st.connection('mydb', type='sql')
df = conn.query('SELECT * FROM users WHERE id = :id',
                params={'id': 1}, ttl=600)    # built-in caching, parameterized
```

```
# .streamlit/config.toml

[theme]
base = "dark"
primaryColor = "#1f77b4"
backgroundColor = "#0e1117"
font = "sans serif"

[server]
port = 8501
maxUploadSize = 200
enableCORS = false

[browser]
gatherUsageStats = false
```

```
st.set_page_config(page_title='Dashboard', page_icon=':chart:',
                  layout='wide', initial_sidebar_state='expanded')
```

`st.set_page_config` must be the **first** Streamlit call in the script, before any other `st.*` command, or it raises.

62. Deployment

Target	Notes
Streamlit Community Cloud	free, GitHub-connected, secrets in the UI
Docker + any VM/PaaS	full control; the usual production route
Hugging Face Spaces	easy for demos and ML models
Behind nginx	needs WebSocket proxying (Upgrade/Connection headers)

```
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
EXPOSE 8501
HEALTHCHECK CMD curl --fail http://localhost:8501/_stcore/health || exit 1
CMD ["streamlit", "run", "app.py", "--server.port=8501", "--server.address=0.0.0.0"]
```

```
location / {
    proxy_pass http://127.0.0.1:8501;
    proxy_http_version 1.1;
    proxy_set_header Upgrade $http_upgrade;           # WebSocket upgrade is REQUIRED
    proxy_set_header Connection "upgrade";
    proxy_set_header Host $host;
    proxy_read_timeout 86400;
}
```

GOTCHA - forgetting the WebSocket headers produces an app that loads the page but never updates. It is the single most common Streamlit deployment bug.

Authentication

Streamlit has no built-in user model. Options: [st.login](#)/OIDC configuration (1.42+), a reverse proxy with SSO in front, [streamlit-authenticator](#), or a simple password gate:

```
def gate():
    if st.session_state.get('auth'): return True
    pw = st.text_input('Password', type='password')
    if pw and hmac.compare_digest(pw, st.secrets['APP_PW']):
        st.session_state.auth = True
        st.rerun()
    return False

if not gate(): st.stop()
```

Use `hmac.compare_digest`, not `==`, to avoid timing attacks - a detail worth mentioning if security comes up.

63. Streamlit vs Flask vs Dash vs Gradio

	Streamlit	Flask	Dash	Gradio
Purpose	data apps/dashboards	general web/API	analytical dashboards	ML model demos
Model	full script rerun	request/response	reactive callbacks	function-to-UI
Front-end code	none	you write it	none (Plotly/React)	none
Control over UI	low	total	medium	low
Best for	internal tools, prototypes	production APIs	enterprise BI	model showcases

INTERVIEW - "When would you NOT use Streamlit?" When you need a public, high-traffic site; fine-grained UI control or a design system; complex routing and SEO; a REST API for other services; or many concurrent users with heavy per-session compute (each session holds server-side state and a WebSocket). In those cases: Flask/FastAPI plus React, or Django.

INTERVIEW - "Can Streamlit and Flask work together?" Yes, and it is a common architecture: Flask serves the REST API and owns the business logic and database; Streamlit is a thin client that calls it with `requests` and renders the results. This keeps the app logic testable and lets other clients reuse the API. That is exactly the capstone in Part V.

64. Streamlit Gotchas Cheat Sheet

Trap	Cause	Fix
Variables reset every click	full script rerun	<code>st.session_state</code>
<code>if st.button()</code> block never persists	buttons are momentary	store the result in state
App is slow on every interaction	no caching	<code>st.cache_data</code> / <code>cache_resource</code>
<code>DuplicateWidgetID</code>	identical widget args	pass distinct <code>key=</code>
<code>UnhashableParamError</code>	unhashable cache arg	underscore-prefix or <code>hash_funcs</code>
Stale data after the source changes	cache with no ttl	<code>ttl=</code> , or <code>.clear()</code>
Cannot modify a widget's state after creation	Streamlit restriction	use <code>on_change</code> callbacks
<code>set_page_config</code> error	not the first <code>st.*</code> call	move it to the top
App loads but never updates in production	WebSocket not proxied	nginx <code>Upgrade</code> headers
All users see the same mutated object	<code>cache_resource</code> is shared	use <code>cache_data</code> , or lock
Chart doesn't refresh	mutating a cached DataFrame	return a copy / cache the derived result

65. Streamlit Interview Drill

Q1. How does Streamlit work under the hood? A Tornado server runs your script per session; the browser holds a WebSocket; each interaction re-executes the script and Streamlit diffs the resulting element tree, sending only changes.

Q2. `cache_data` vs `cache_resource`? Data versus global objects; copy versus shared instance; picklable versus not. Give the connection/model example.

- Q3. What is `st.session_state` and what are its limits?** Per-session server-side dict that survives reruns but not refreshes or restarts; not shared across users; not persistent storage.
- Q4. Why does my variable reset?** Because the entire script reruns. Persist in session state.
- Q5. When do you use `st.form`?** To batch several inputs into one submission and one rerun.
- Q6. How do you build a multipage app?** `st.navigation` with `st.Page` objects, or a `pages/` directory. Share state via session state or query params.
- Q7. How do you handle authentication?** Not built in historically; use OIDC configuration, a reverse proxy with SSO, `streamlit-authenticator`, or a hashed secret compared with `hmac.compare_digest`.
- Q8. How would you display a 10-million-row table?** You would not. Aggregate or sample server-side, push filtering into SQL, paginate, and cache the reduced result.
- Q9. How do you make one component refresh without rerunning everything?** `@st.fragment`, optionally with `run_every`.
- Q10. How do you deploy Streamlit securely?** Container image, secrets from the environment or `secrets.toml` (never committed), HTTPS with WebSocket proxying, authentication in front, resource limits, and a health check on `/_stcore/health`.
- Q11. Is Streamlit suitable for production?** For internal tools, dashboards, and demos - yes, widely. For public high-traffic consumer apps - usually not, because of per-session server state, limited UI control, and no SEO.
- Q12. How do you pass data between Streamlit and a backend?** HTTP calls to a REST API (`requests`), a shared database via `st.connection`, or a message queue. Keep business logic in the backend, not in the Streamlit script.

PART V - Capstone: One Project, All Four Libraries

66. What You Are Building

A **Sales Analytics Platform**:

```
raw CSV  ->  [NumPy + Pandas ETL]  ->  SQLite/Postgres
                                     |
                                     [Flask REST API]
                                     |
                                     [Streamlit dashboard]
```

Why this shape? Because it demonstrates the thing interviewers are actually probing for: that you can put a boundary between *data logic*, *service logic*, and *presentation*, instead of writing one 800-line script.

Project layout:

```
sales-platform/
  etl/
    clean.py          # pandas + numpy transformation
  api/
    __init__.py       # create_app factory
    models.py
    routes.py
    extensions.py
  dashboard/
    app.py            # streamlit client
  config.py
  wsgi.py
  requirements.txt
  tests/
    test_etl.py
    test_api.py
```

67. Stage 1 - The ETL Layer (NumPy + Pandas)

```

# etl/clean.py
import numpy as np
import pandas as pd

RAW_DTYPES = {'order_id': 'int64', 'customer_id': 'int32',
              'region': 'category', 'quantity': 'int16', 'unit_price': 'float32'}

def load_raw(path: str) -> pd.DataFrame:
    return pd.read_csv(
        path,
        usecols=list(RAW_DTYPES) + ['order_date', 'discount'],
        dtype=RAW_DTYPES,
        parse_dates=['order_date'],
        na_values=['', 'NA', 'null', '-'],
    )

def clean(df: pd.DataFrame) -> pd.DataFrame:
    df = df.copy()

    # 1. Drop exact duplicates, then keep the latest record per order
    df = (df.sort_values('order_date')
          .drop_duplicates(subset=['order_id'], keep='last'))

    # 2. Impute: discount missing means no discount; price by region median
    df['discount'] = df['discount'].fillna(0.0)
    df['unit_price'] = df['unit_price'].fillna(
        df.groupby('region', observed=True)['unit_price'].transform('median')
    )

    # 3. Remove impossible values instead of silently keeping them
    df = df[(df.quantity > 0) & (df.unit_price > 0)]
    df = df[df.discount.between(0, 0.9)]

    # 4. Derived columns - vectorized, no apply
    df['gross'] = df.quantity * df.unit_price
    df['net'] = df.gross * (1 - df.discount)
    df['month'] = df.order_date.dt.to_period('M').dt.to_timestamp()
    df['weekday'] = df.order_date.dt.day_name()

    # 5. Flag outliers with the IQR rule (NumPy)
    q1, q3 = np.percentile(df.net, [25, 75])
    iqr = q3 - q1
    df['is_outlier'] = (df.net < q1 - 1.5*iqr) | (df.net > q3 + 1.5*iqr)

    # 6. Rank within region using a window-style transform
    df['region_rank'] = (df.groupby('region', observed=True)['net']
                        .rank(method='dense', ascending=False)
                        .astype('int32'))

    return df

def monthly_summary(df: pd.DataFrame) -> pd.DataFrame:
    out = (df.groupby(['month', 'region'], observed=True, as_index=False)
          .agg(revenue=('net', 'sum'),
              orders=('order_id', 'size'),
              avg_order=('net', 'mean'),
              units=('quantity', 'sum')))
    out['mom_growth'] = (out.sort_values('month')
                        .groupby('region', observed=True)['revenue']
                        .pct_change())

    return out

```

```

if __name__ == '__main__':
    raw = load_raw('data/orders.csv')
    tidy = clean(raw)
    tidy.to_parquet('data/orders_clean.parquet', index=False)
    monthly_summary(tidy).to_parquet('data/monthly.parquet', index=False)
    print(tidy.info(memory_usage='deep'))

```

Everything worth defending in an interview is in this file: dtype control at read time, deduplication with an explicit `keep` rule, group-wise imputation, validity filtering, vectorized derived columns, an IQR outlier rule via NumPy, a window-function-style rank, and a Parquet output.

68. Stage 2 - The Flask API

```

# api/extensions.py
from flask_sqlalchemy import SQLAlchemy
db = SQLAlchemy()

```

```

# api/models.py
from .extensions import db

class Order(db.Model):
    __tablename__ = 'orders'
    order_id = db.Column(db.Integer, primary_key=True)
    customer_id = db.Column(db.Integer, index=True, nullable=False)
    region = db.Column(db.String(32), index=True, nullable=False)
    order_date = db.Column(db.Date, index=True, nullable=False)
    quantity = db.Column(db.Integer, nullable=False)
    unit_price = db.Column(db.Float, nullable=False)
    net = db.Column(db.Float, nullable=False)

    def to_dict(self):
        return {'order_id': self.order_id, 'customer_id': self.customer_id,
                'region': self.region, 'order_date': self.order_date.isoformat(),
                'quantity': self.quantity, 'unit_price': self.unit_price,
                'net': round(self.net, 2)}

```

```

# api/routes.py
from flask import Blueprint, request, abort
from sqlalchemy import func
from .models import Order
from .extensions import db

bp = Blueprint('api', __name__)

@bp.get('/health')
def health():
    return {'status': 'ok'}

@bp.get('/orders')
def orders():
    page = request.args.get('page', 1, type=int)
    per = min(request.args.get('per_page', 50, type=int), 200)
    region = request.args.get('region')
    start = request.args.get('start')
    end = request.args.get('end')

    q = Order.query
    if region: q = q.filter(Order.region == region)
    if start: q = q.filter(Order.order_date >= start)
    if end: q = q.filter(Order.order_date <= end)

    p = q.order_by(Order.order_date.desc()).paginate(
        page=page, per_page=per, error_out=False)
    return {'data': [o.to_dict() for o in p.items],
            'meta': {'page': p.page, 'pages': p.pages, 'total': p.total}}

@bp.get('/summary')
def summary():
    """Aggregate in the DATABASE, not in pandas. This is the key design call."""
    rows = (db.session.query(
        Order.region,
        func.strftime('%Y-%m', Order.order_date).label('month'),
        func.sum(Order.net).label('revenue'),
        func.count(Order.order_id).label('orders'),
        func.avg(Order.net).label('avg_order'))
        .group_by('region', 'month')
        .order_by('month')
        .all())
    return {'data': [{ 'region': r.region, 'month': r.month,
                        'revenue': round(r.revenue, 2), 'orders': r.orders,
                        'avg_order': round(r.avg_order, 2)} for r in rows]}

@bp.get('/regions')
def regions():
    return {'data': [r[0] for r in db.session.query(Order.region).distinct()]}

@bp.get('/orders/<int:order_id>')
def one(order_id):
    o = db.session.get(Order, order_id) or abort(404)
    return o.to_dict()

```

```

# api/__init__.py
from flask import Flask
from .extensions import db

def create_app(config_object='config.Dev'):
    app = Flask(__name__)
    app.config.from_object(config_object)
    db.init_app(app)

    from .routes import bp
    app.register_blueprint(bp, url_prefix='/api/v1')

    @app.errorhandler(404)
    def nf(e): return {'error': 'not found'}, 404

    @app.errorhandler(Exception)
    def boom(e):
        db.session.rollback()
        app.logger.exception('unhandled')
        return {'error': 'internal server error'}, 500

    return app

```

```

# load the cleaned parquet into the database, once
import pandas as pd
from api import create_app
from api.extensions import db

app = create_app()
with app.app_context():
    db.create_all()
    df = pd.read_parquet('data/orders_clean.parquet')
    cols = ['order_id', 'customer_id', 'region', 'order_date',
            'quantity', 'unit_price', 'net']
    df[cols].to_sql('orders', db.engine, if_exists='replace', index=False)

```

The design decision to be able to defend: the `/summary` endpoint aggregates in SQL rather than pulling every row into pandas. Pandas is the right tool for exploratory and complex transformations; a database is the right tool for filtering and aggregating at serving time, because it has indexes and does not ship millions of rows over the wire.

69. Stage 3 - The Streamlit Dashboard

```

# dashboard/app.py
import os
import pandas as pd
import requests
import streamlit as st

API = os.environ.get('API_URL', 'http://localhost:8000/api/v1')

st.set_page_config(page_title='Sales Analytics', layout='wide')

@st.cache_data(ttl=300, show_spinner='Fetching summary...')
def get_summary() -> pd.DataFrame:
    r = requests.get(f'{API}/summary', timeout=10)
    r.raise_for_status()
    df = pd.DataFrame(r.json()['data'])
    df['month'] = pd.to_datetime(df['month'])
    return df

@st.cache_data(ttl=300)
def get_regions() -> list[str]:
    return requests.get(f'{API}/regions', timeout=10).json()['data']

@st.cache_data(ttl=60)
def get_orders(region: str | None, page: int) -> dict:
    params = {'page': page, 'per_page': 50}
    if region and region != 'All':
        params['region'] = region
    return requests.get(f'{API}/orders', params=params, timeout=10).json()

st.title('Sales Analytics')

try:
    summary = get_summary()
    regions = ['All'] + get_regions()
except requests.RequestException as e:
    st.error(f'API unavailable: {e}')
    st.stop()

with st.sidebar:
    st.header('Filters')
    region = st.selectbox('Region', regions)
    months = st.slider('Months to show', 3, 24, 12)

view = summary if region == 'All' else summary[summary.region == region]
view = view[view.month >= view.month.max() - pd.DateOffset(months=months)]

c1, c2, c3, c4 = st.columns(4)
rev = view.revenue.sum()
prev = summary[summary.month < view.month.min()].revenue.sum()
c1.metric('Revenue', f'${rev:,.0f}',
          delta=f'{{(rev/prev - 1):.1%}}' if prev else None)
c2.metric('Orders', f'{{int(view.orders.sum()):,}}')
c3.metric('Avg order', f'${view.revenue.sum()/max(view.orders.sum(),1):,.2f}')
c4.metric('Regions', view.region.nunique())

tab1, tab2, tab3 = st.tabs(['Trend', 'Breakdown', 'Orders'])

with tab1:
    pivot = view.pivot_table(index='month', columns='region',
                              values='revenue', aggfunc='sum')
    st.line_chart(pivot)

```

```

with tab2:
    by_region = (view.groupby('region', as_index=False)['revenue']
                  .sum().sort_values('revenue', ascending=False))
    st.bar_chart(by_region, x='region', y='revenue')
    st.dataframe(by_region, hide_index=True, use_container_width=True)

with tab3:
    st.session_state.setdefault('page', 1)
    payload = get_orders(region, st.session_state.page)
    st.dataframe(pd.DataFrame(payload['data']), hide_index=True,
                  use_container_width=True)
    a, b, c = st.columns([1, 1, 6])
    if a.button('Prev', disabled=st.session_state.page <= 1):
        st.session_state.page -= 1; st.rerun()
    if b.button('Next',
                disabled=st.session_state.page >= payload['meta']['pages']):
        st.session_state.page += 1; st.rerun()
    c.caption(f"Page {payload['meta']['page']} of {payload['meta']['pages']}")

st.download_button('Download summary CSV',
                    view.to_csv(index=False).encode(),
                    'summary.csv', 'text/csv')

```

Every Streamlit concept from Part IV appears here: `set_page_config` first, `cache_data` with a TTL on every network call, session state for pagination, columns/tabs/sidebar layout, `st.stop()` on a failed dependency, and a download button.

70. Stage 4 - Tests

```

# tests/test_etl.py
import pandas as pd
from etl.clean import clean

def test_clean_removes_invalid_rows():
    raw = pd.DataFrame({
        'order_id': [1, 2, 3], 'customer_id': [1, 1, 2],
        'region': pd.Categorical(['N', 'N', 'S']),
        'order_date': pd.to_datetime(['2024-01-01']*3),
        'quantity': [1, 0, 2], 'unit_price': [10.0, 5.0, 7.0],
        'discount': [0.0, 0.0, None]})
    out = clean(raw)
    assert len(out) == 2 # the quantity==0 row is dropped
    assert out.discount.isna().sum() == 0
    assert (out.net <= out.gross).all()

```

```
# tests/test_api.py
import pytest
from api import create_app
from api.extensions import db

@pytest.fixture
def client():
    app = create_app('config.Test')
    with app.app_context():
        db.create_all()
        yield app.test_client()
        db.drop_all()

def test_health(client):
    assert client.get('/api/v1/health').get_json() == {'status': 'ok'}

def test_orders_empty(client):
    body = client.get('/api/v1/orders').get_json()
    assert body['data'] == [] and body['meta']['total'] == 0

def test_404(client):
    assert client.get('/api/v1/orders/999').status_code == 404
```

71. Running and Shipping It

```
python -m etl.clean           # build the parquet files
python load_db.py             # seed the database
gunicorn -w 4 -b 0.0.0.0:8000 'wsgi:app' # API
streamlit run dashboard/app.py --server.port 8501 # dashboard
pytest -q                     # tests
```

```
# docker-compose.yml
services:
  api:
    build: .
    command: gunicorn -w 4 -b 0.0.0.0:8000 wsgi:app
    environment: [DATABASE_URL=sqlite:///data/app.db]
    ports: ["8000:8000"]
    volumes: [".:/data:/data"]
  dashboard:
    build: .
    command: streamlit run dashboard/app.py --server.address=0.0.0.0
    environment: [API_URL=http://api:8000/api/v1]
    ports: ["8501:8501"]
    depends_on: [api]
```

Extensions to make it interview-grade

1. Add JWT auth to the API and a login gate on the dashboard.
2. Add Redis caching on [/summary](#) and compare latency before and after.
3. Replace SQLite with Postgres and add indexes on ([region](#), [order_date](#)); show the [EXPLAIN](#) output before and after.
4. Move the ETL into a Celery beat schedule so data refreshes nightly.
5. Add an anomaly-detection column using a rolling z-score and surface it in the dashboard.

6. Add CI: pytest plus `ruff` plus a Docker build on every push.

How to talk about it

Interviewers do not want a feature tour. They want the decisions. Rehearse these four sentences:

- *"I aggregate in SQL rather than pandas at serving time because the database has indexes and I avoid shipping millions of rows over the network."*
- *"The dashboard is a thin client over the API, so the business logic is testable and reusable by other consumers."*
- *"I control dtypes at read time; the cleaned frame is about a fifth of the raw memory footprint."*
- *"Every network call in Streamlit is cached with a TTL, because the script reruns from the top on every interaction."*

PART VI - Interview Preparation

72. How These Interviews Are Actually Structured

Most data/backend Python interviews have four recognisable stages. Knowing which stage you are in tells you how to answer.

Stage	What they test	How to answer
Phone screen	vocabulary, breadth	short, correct, name the concept
Live coding	fluency, debugging	narrate your thinking, test edge cases
Deep dive	depth on one topic	give the mechanism, not just the API
System/design	judgement, trade-offs	state constraints, then choose, then name the cost

Three habits that raise your score independent of knowledge:

1. **Ask about the data before you code.** Size, cardinality, nullability, ordering guarantees. Interviewers score this explicitly.
2. **Name the trade-off you are accepting.** "I'll use a dict lookup - $O(1)$ at the cost of memory" beats a silent correct answer.
3. **Say "I don't know, here's how I'd find out."** It is scored higher than a confident wrong answer, every time.

73. Rapid-Fire Bank: Python Fundamentals

These come up alongside the libraries in nearly every interview.

1. **List vs tuple?** Mutable vs immutable; tuples are hashable and can be dict keys; tuples are slightly smaller and faster to construct.
2. **Shallow vs deep copy?** `copy.copy` copies the container but shares nested objects; `copy.deepcopy` recursively copies everything. Relevant whenever you mutate a nested structure that a caller also holds.
3. **Mutable default argument trap?** `def f(x=[])` creates the list once at function definition and reuses it across calls. Use `x=None` and assign inside.
4. **is vs ==?** Identity vs equality. Use `is` only for `None`, `True`, `False`, and sentinels.
5. **What is a generator?** A lazy iterator produced by `yield`; it holds one item in memory at a time. Essential for streaming large files.
6. **What is a decorator?** A callable that takes a function and returns a replacement, applied with `@`. Use `functools.wraps` to preserve metadata.
7. ***args / **kwargs?** Variadic positional and keyword arguments; collection at definition, unpacking at call.
8. **What is the GIL?** The Global Interpreter Lock allows only one thread to execute Python bytecode at a time in CPython. Threads still help I/O-bound work (the lock is released during I/O); CPU-bound work needs `multiprocessing`, a C extension that releases the GIL (NumPy does), or a free-threaded build.
9. **List comprehension vs map/filter?** Comprehensions are more readable and usually as fast; generator expressions avoid materializing the list.
10. **__str__ vs __repr__?** `str` is for users, `repr` for developers and debugging; `repr` should ideally be unambiguous and reconstructable.
11. **Context managers?** Objects implementing `__enter__` / `__exit__`, used with `with`, guaranteeing cleanup even on exceptions. `contextlib.contextmanager` builds one from a generator.

- 12. How does Python manage memory?** Reference counting plus a cyclic garbage collector for reference cycles; small-object allocation via pymalloc arenas.
- 13. @staticmethod vs @classmethod?** Static receives nothing implicit; class receives the class (`cls`) and is the idiom for alternative constructors.
- 14. What is `__slots__`?** Declares a fixed attribute set, removing the per-instance `__dict__` - big memory savings for many small objects.
- 15. Explain time complexity of Python built-ins.** list index $O(1)$, list append amortized $O(1)$, list insert/remove $O(n)$, `in` on list $O(n)$, `in` on set/dict $O(1)$ average, sort $O(n \log n)$.
-

74. Rapid-Fire Bank: NumPy (30 questions)

- 1. Why NumPy over lists?** Contiguous typed memory, C loops, no per-element dispatch, SIMD.
 - 2. What are strides?** Byte steps per axis; they make transpose and slicing free.
 - 3. View or copy for `a[1:3]`?** View.
 - 4. View or copy for `a[[1,3]]`?** Copy (fancy indexing).
 - 5. `ravel` vs `flatten`?** View when possible vs always copy.
 - 6. Broadcasting rules?** Right-align; dims must be equal or 1.
 - 7. What does `keepdims=True` do?** Retains reduced axes as length 1 so the result broadcasts back against the original.
 - 8. What does `axis=0` mean?** That axis is collapsed; for a 2-D array it means a per-column result.
 - 9. `np.dot` vs `*`?** Matrix product vs elementwise.
 - 10. Why `solve` over `inv`?** Faster and numerically more stable.
 - 11. How to test NaN?** `np.isnan`; `== np.nan` is always False.
 - 12. `np.nan` in an int array?** Impossible - the array upcasts to float.
 - 13. `a.sort()` vs `np.sort(a)`?** In place returning None vs sorted copy.
 - 14. Top-k without full sort?** `np.argpartition`, $O(n)$.
 - 15. Sort rows by a column?** `a[a[:,k].argsort()]`.
 - 16. Multi-key sort?** `np.lexsort`, last key is primary.
 - 17. What is a ufunc?** A vectorized elementwise C function with `reduce`, `accumulate`, `outer`, and `at` methods.
 - 18. Does `np.vectorize` speed things up?** No, it loops in Python.
 - 19. What does `out=` give you?** Avoids allocating a new output array.
 - 20. How to one-hot encode?** `np.eye(n)[labels]`.
 - 21. How to seed reproducibly?** `np.random.default_rng(seed)`.
 - 22. C order vs F order?** Row-major vs column-major; affects cache behaviour and whether reshape can avoid a copy.
 - 23. What is `np.newaxis`?** Inserts an axis of length 1; same as `None`.
 - 24. Concatenate vs stack?** Existing axis vs new axis.
 - 25. Why is appending in a loop bad?** Reallocation and copying, $O(n^2)$.
 - 26. Memory for arrays bigger than RAM?** `np.memmap`.
 - 27. `np.where` with one argument?** Returns index arrays of the True positions.
 - 28. How do you compare floats?** `np.isclose` / `np.allclose`.
 - 29. What is `np.einsum`?** Index-notation contractions; expresses matmul, trace, transpose, and batched operations in one string.
 - 30. Why does `np.std` differ from `Series.std`?** `ddof=0` vs `ddof=1`.
-

75. Rapid-Fire Bank: Pandas (45 questions)

1. **Series vs DataFrame?** 1-D labelled array vs 2-D table of aligned Series.
2. **What does the Index do?** Labels, alignment, fast lookup.
3. **loc vs iloc?** Label (end-inclusive) vs position (end-exclusive).
4. **at vs loc?** Scalar fast path vs general.
5. **What is chained indexing?** Two sequential `[]` operations; the first may copy, so assignment may be lost.
6. **How to fix SettingWithCopyWarning?** One `.loc[rows, cols] = value`.
7. **What is Copy-on-Write?** Pandas 3.0 semantics where results behave as copies; chained assignment never propagates.
8. **How to detect nulls?** `isna()`; sum or mean it per column.
9. **Why did my int column become float?** `np.nan` is a float.
10. **What is Int64 vs int64?** Nullable masked dtype vs NumPy dtype.
11. **dropna parameters?** `how`, `subset`, `thresh`, `axis`.
12. **Fill strategies?** Constant, statistic, ffill/bfill, interpolate, group-wise, model-based.
13. **apply vs map vs applymap?** Axis-wise vs Series-elementwise vs DataFrame-elementwise.
14. **Why avoid iterrows?** Builds a Series per row; use vectorization or `itertuples`.
15. **agg vs transform?** Collapses to one row per group vs returns original-shaped, alignable result.
16. **groupby.filter?** Keeps or drops whole groups based on a predicate.
17. **size vs count on groupby?** Rows including NaN vs non-null per column.
18. **How to keep the group key as a column?** `as_index=False` or `reset_index()`.
19. **Named aggregation syntax?** `.agg(new=('col', 'func'))`.
20. **merge vs join vs concat?** Key join vs index join vs stacking.
21. **Join types?** inner, left, right, outer, cross.
22. **Why did my merge add rows?** Duplicate keys on the right - fan-out.
23. **How do you prevent that?** `validate='m:1'` and `indicator=True`.
24. **What is an anti-join in pandas?** Left merge with `indicator=True`, then filter `_merge == 'left_only'`.
25. **merge_asof?** Nearest-preceding-key join for time series.
26. **pivot vs pivot_table?** No aggregation (raises on duplicates) vs aggregates.
27. **melt?** Wide to long; the inverse of pivot.
28. **stack vs unstack?** Columns to index level vs index level to columns.
29. **crosstab?** Frequency table with normalization and margins.
30. **get_dummies?** One-hot encoding; use `drop_first` to avoid the dummy variable trap.
31. **cut vs qcut?** Equal-width bins vs equal-frequency (quantile) bins.
32. **How do you rank within groups?** `groupby().rank(method=...)`.
33. **shift / diff / pct_change?** Lag, first difference, relative change.
34. **rolling vs expanding vs ewm?** Fixed window, cumulative, exponentially weighted.
35. **resample vs groupby?** Time-bin groupby requiring a `DatetimeIndex`; generates empty bins.
36. **How to handle time zones?** `tz_localize` to attach, `tz_convert` to change; store UTC, display local.
37. **How to parse messy dates?** Explicit `format=`, `errors='coerce'`, then inspect the `NaT` rate.
38. **What is a MultiIndex and its main trap?** Hierarchical labels; must be sorted before slicing.
39. **How do you reduce memory?** Downcast, `category`, `usecols`, `string[pyarrow]`, Parquet.
40. **How to read a file larger than RAM?** `chunksize`, column pruning, Parquet, or Dask/Polars/DuckDB.
41. **CSV vs Parquet?** Row text vs columnar typed compressed; Parquet supports column pruning and predicate pushdown.
42. **Does inplace=True save memory?** Generally no, and it breaks chaining.
43. **Why is query sometimes faster?** numexpr evaluation avoids intermediate temporaries on large frames.

44. **What is method chaining and why use it?** A pipeline of pure transformations; avoids intermediates and copy warnings.
 45. **When would you leave pandas?** When data exceeds memory, when you need multi-core or distributed execution, or when a SQL engine already has the data and can aggregate it in place.
-

76. Rapid-Fire Bank: Flask (40 questions)

1. **What is Flask?** WSGI micro-framework on Werkzeug and Jinja2.
2. **What is WSGI?** The sync Python web server/application interface.
3. **WSGI vs ASGI?** Sync one-request-per-worker vs async event-loop capable.
4. **Why not `app.run()` in production?** Development server: no process management, poor concurrency, insecure with debug on.
5. **What does `@app.route` do?** Adds a rule to the URL map bound to an endpoint.
6. **URL converters?** `int`, `float`, `string`, `path`, `uuid`.
7. **Trailing slash behaviour?** `/x/` redirects from `/x`; `/x` 404s on `/x/`.
8. **Why `url_for`?** Refactor-safe, handles quoting, prefixes, and `_external`.
9. **Application context vs request context?** `current_app/g` vs `request/session`.
10. **What is `g`?** Per-request scratch namespace.
11. **"Working outside of application context" - why?** No context pushed; use `with app.app_context():`.
12. **Why are the globals thread-safe?** They are context-local proxies backed by `contextvars`.
13. **What is a Blueprint?** A registerable group of routes and handlers with an optional URL prefix.
14. **What is the app factory pattern?** `create_app(config)` for deferred, parameterized construction; avoids circular imports and enables test config.
15. **Where does the session live?** A signed (not encrypted) client cookie.
16. **What signs it?** `SECRET_KEY`.
17. **Session vs JWT?** Server state and easy revocation vs stateless and easy scaling.
18. **How do you hash passwords?** `generate_password_hash` / a slow salted KDF.
19. **401 vs 403?** Unauthenticated vs unauthorized.
20. **200 vs 201 vs 204?** OK, Created (with Location), No Content.
21. **422 vs 400?** Well-formed but semantically invalid vs malformed.
22. **Which verbs are idempotent?** GET, PUT, DELETE, HEAD, OPTIONS (not POST).
23. **What makes REST stateless?** No server-side per-client state between requests; every request is self-contained.
24. **How do you validate input?** Marshmallow/Pydantic schemas at the boundary.
25. **How do you prevent SQL injection?** ORM or parameterized queries.
26. **How does Flask prevent XSS?** Jinja2 autoescaping; add CSP.
27. **What is CSRF and the defence?** Forged authenticated requests; per-session tokens plus `SameSite`.
28. **What is CORS?** Server-declared permission for cross-origin reads; enforced by the browser only.
29. **`before_request` vs `after_request` vs `teardown_request`?** Pre-dispatch (can short-circuit), post-response (must return it, skipped on exception), always-runs cleanup.
30. **How do you add real middleware?** Wrap `app.wsgi_app`.
31. **What is `ProxyFix` for?** Trusting `X-Forwarded-*` behind a reverse proxy.
32. **What is the N+1 problem?** One query per parent row; fix with eager loading.
33. **How do you do migrations?** Flask-Migrate/Alembic; `flask db migrate` then `upgrade`, run as a deploy step.
34. **How do you test?** `app.test_client()`, pytest fixtures, test config, isolated database.
35. **How do you handle a 30-second job?** Background queue, return 202 with a task id.
36. **Does Flask support async views?** Yes, but they run in a worker thread; it is not ASGI concurrency.

37. **How many unicorn workers?** Roughly `2 * cores + 1`; threads or gevent for I/O-bound loads.
 38. **Why nginx in front?** TLS, static files, buffering, compression, LB.
 39. **How do you version an API?** URL prefix via blueprints.
 40. **Flask vs FastAPI vs Django?** Control vs typed-async-with-docs vs batteries-included.
-

77. Rapid-Fire Bank: Streamlit (25 questions)

1. **Execution model?** Full top-to-bottom script rerun per interaction.
 2. **Why did my variable reset?** Because of that rerun; use session state.
 3. **What is `st.session_state`?** Per-session server-side dict surviving reruns.
 4. **Is it shared across users?** No.
 5. **Does it survive a refresh?** No - use a database or query params.
 6. **`cache_data` vs `cache_resource`?** Copies of data vs one shared object.
 7. **Which for a DB connection?** `cache_resource`.
 8. **How is the cache key made?** Hash of the function body plus arguments.
 9. **How do you exclude an argument?** Prefix it with `_`, or use `hash_funcs`.
 10. **How do you invalidate?** `ttl`, `func.clear()`, `st.cache_data.clear()`.
 11. **Why is `if st.button()` tricky?** It is True for one rerun only.
 12. **Why use `st.form`?** Batch several widgets into a single submit rerun.
 13. **What can't go inside a form?** `st.button`, and `on_change` callbacks.
 14. **What is a fragment?** A function that reruns independently of the script.
 15. **`st.rerun` vs `st.stop`?** Restart from the top vs halt the script here.
 16. **`DuplicateWidgetID` fix?** Distinct `key=`.
 17. **When do callbacks run?** Before the script rerun.
 18. **How do you build multipage apps?** `st.navigation` + `st.Page`, or the `pages/` directory.
 19. **How do you share state between pages?** Session state; query params for shareable/refresh-safe state.
 20. **Where do secrets go?** `.streamlit/secrets.toml` or environment variables, never in git.
 21. **What must be the first `st.*` call?** `st.set_page_config`.
 22. **Most common deployment bug?** Missing WebSocket proxy headers in nginx.
 23. **How do you show a very large table?** Aggregate/paginate server-side.
 24. **Streamlit or Flask?** Internal dashboard vs public API/site; often both, with Streamlit as a thin client.
 25. **Authentication options?** Built-in OIDC login, SSO proxy, `streamlit-authenticator`, or a hashed password gate with `hmac.compare_digest`.
-

78. Scenario and Design Questions

These are the differentiators. Answer with constraints, then a choice, then the cost of that choice.

"A pandas script that used to take 2 minutes now takes 40. What do you do?" Profile before guessing: time each stage. Then check, in order - did the data volume grow, or the cardinality? Is there an `apply` or `iterrows` in the hot path? Did a merge start fanning out (compare row counts before/after)? Are dtypes still narrow, or did something become `object`? Is the machine swapping (`memory_usage(deep=True)` versus available RAM)? Fixes, in order of payoff: vectorize, fix the join, narrow dtypes, push filtering to the source, then consider Polars/DuckDB.

"Design an API that serves aggregated analytics over 500M rows." Constraints first: query latency target, freshness requirement, cardinality of the group-by keys. Then: do not aggregate at request time over raw rows - precompute rollups on a schedule into a summary table, index on the filter columns, cache hot responses in Redis with a TTL, paginate with keyset pagination, and return a stable

envelope. Mention that a columnar store (ClickHouse, BigQuery, DuckDB) is the right home for this, not row-oriented Postgres, if scans dominate.

"Users report the dashboard is slow after you added a filter." Each filter change triggers a full Streamlit rerun. If the data fetch is not cached, you re-query on every keystroke. Fix: `cache_data` with a TTL keyed on the filter values, push the filter into the SQL `WHERE` clause rather than loading everything and filtering in pandas, wrap multiple filters in a form so one submit causes one rerun, and isolate the live region in a fragment.

"How would you handle a CSV with inconsistent schemas across files?" Read each with `dtype=str` first to avoid inference surprises, normalize column names (`str.lower().str.strip()`), map aliases to a canonical schema, validate required columns and fail loudly with the filename, coerce types explicitly with `errors='coerce'`, record the coercion failure rate per column, then `pd.concat` with `join='outer'` and an origin column for traceability.

"Your API returns 500s intermittently under load. Debug it." Look at logs and traces first: is it one endpoint or all? Common causes - database connection pool exhaustion (pool size below worker count x threads), a slow query holding connections, a memory leak in a `cache_resource` object, a worker timeout killing long requests, or an unhandled exception in a code path only hit with certain data. Instrument with request ids, latency percentiles, and pool metrics; add a timeout and circuit breaker on outbound calls.

"How do you make results reproducible?" Pin versions in a lock file, seed every RNG explicitly and pass generators rather than using globals, fix the ordering of any non-deterministic operation (`sort_values` before `head`), record the data snapshot identifier, and containerize the runtime.

"Walk me through cleaning a dataset you have never seen." `df.info()`, `describe(include='all')`, null rate per column, cardinality per column, and a sample of raw rows. Then: fix dtypes, standardize text, resolve duplicates with an explicit rule, decide missingness treatment per column with a reason, validate ranges and referential integrity, flag rather than silently delete outliers, and write the cleaned output with a schema and a row-count reconciliation against the input.

79. Cheat Sheets

NumPy one-page

```
np.array / zeros / ones / full / arange / linspace / eye
a.shape .dtype .ndim .size .strides .nbytes .T
a.reshape(-1,1) .ravel() .flatten() np.expand_dims np.squeeze
a[1:3]          # view      a[[1,3]] / a[a>2] # copy
a.sum(axis=0, keepdims=True) .mean .std(ddof=) .min .argmax .cumsum
np.nansum / nanmean / isnan / isclose / allclose
np.where(c,x,y) / np.select / np.clip / np.unique(return_counts=True)
np.concatenate / vstack / hstack / stack / split / tile / repeat
np.sort / argsort / lexsort / argpartition / searchsorted
A @ B / np.linalg.solve / inv / det / eig / svd / norm / einsum
rng = np.random.default_rng(0); rng.normal / integers / choice / shuffle
```

Pandas one-page

```
pd.read_csv(usecols=, dtype=, parse_dates=, chunksize=, na_values=)
df.info() .describe() .isna().sum() .nunique() .memory_usage(deep=True)
df.loc[rows, cols]      # label, inclusive   df.iloc[i, j]   # position
df.query('a > @x')  df.assign(b=lambda d: ...)  df.pipe(fn)
df.astype('category')  pd.to_numeric(downcast=)  pd.to_datetime(errors='coerce')
df.fillna / ffill / bfill / dropna(subset=, how=, thresh=) / interpolate
df.groupby(k).agg(new=('c', 'sum')) / .transform('mean') / .filter(fn) / .size()
pd.merge(l, r, on=, how=, validate=, indicator=, suffixes=)
pd.concat([..], axis=, ignore_index=, join=)
df.pivot_table(index=, columns=, values=, aggfunc=) / melt / stack / unstack
df.sort_values([..], ascending=[..]) / drop_duplicates(subset=, keep=)
s.str.contains(na=False) / .extract / .split(expand=True)
s.dt.year / .to_period('M') / tz_localize / tz_convert
ts.resample('ME').sum() / .rolling(7, min_periods=1).mean() / .shift / .pct_change
pd.cut / qcut / get_dummies / crosstab / Grouper(freq=)
```

Flask one-page

```
app = Flask(__name__); create_app() factory; Blueprint(name, __name__)
@app.route('/x/<int:i>', methods=['GET', 'POST'])  @app.get @app.post
request.args / .form / .get_json() / .files / .headers / .cookies
return dict | jsonify() | (body, status, headers) | redirect(url_for(...))
render_template('t.html', **ctx)   {% extends %} {% block %} {{ x|filter }}
g / current_app / session  with app.app_context(): ...
@app.before_request  @app.after_request  @app.teardown_appcontext
@app.errorhandler(404 | HTTPException | Exception)
db = SQLAlchemy(); db.init_app(app); db.session.add/commit/rollback
flask db migrate -m "" ; flask db upgrade
app.test_client()  pytest fixtures  config.Test
gunicorn -w 4 -b 0.0.0.0:8000 wsgi:app  + nginx + ProxyFix + HTTPS
```

Streamlit one-page

```
st.set_page_config(...)      # MUST be first
st.title/header/write/markdown/metric/dataframe/data_editor/json
st.button/checkbox/radio/selectbox/multiselect/slider/text_input/file_uploader
st.session_state['k']        # persists across reruns, per session
@st.cache_data(ttl=)          # data, returns a copy
@st.cache_resource            # connections/models, shared instance
st.columns / tabs / sidebar / expander / container / empty / popover / dialog
with st.form('f'): ... st.form_submit_button()
@st.fragment(run_every='5s')  st.rerun()  st.stop()
st.navigation([st.Page(...)]).run()  st.query_params  st.switch_page
st.connection('db', type='sql').query(sql, params=, ttl=)
st.secrets['KEY']  .streamlit/config.toml  streamlit run app.py
```

80. A 30-Day Study Plan

Days	Focus	Deliverable
1-2	NumPy arrays, dtypes, indexing, views vs copies	20 exercises, no docs
3-4	Broadcasting, ufuncs, aggregations, axis	implement k-means by hand
5	Linear algebra, random, sorting, performance	vectorize a loop-based script
6-8	Pandas core: IO, selection, dtypes, missing data	profile a real messy CSV
9-11	GroupBy, merge, reshape	rebuild 10 SQL queries in pandas
12-13	Time series, strings, MultiIndex	cohort + retention analysis
14	Pandas performance and Copy-on-Write	cut a frame's memory by 70%
15	Review + self-test	answer Parts 74-75 aloud
16-17	Flask routing, requests, responses, templates	a CRUD app with Jinja
18-19	Contexts, factory, blueprints, config	restructure it into a factory
20-21	SQLAlchemy, migrations, validation	a versioned REST API
22	Auth, errors, logging, security	JWT + rate limiting + error envelope
23	Testing and deployment	pytest suite + Dockerfile + gunicorn
24-25	Streamlit basics, widgets, layout	a filterable dashboard
26	Session state, caching, forms, fragments	make it fast; measure before/after
27-29	Capstone (Part V)	ETL + API + dashboard, dockerized
30	Mock interview	explain the capstone in 5 minutes

The week-before checklist

- Can you explain view vs copy, broadcasting, and `keepdims` without notes?
- Can you write a groupby-transform and a merge with `validate=` from memory?
- Can you draw the Flask request lifecycle including both contexts?
- Can you state the `cache_data/cache_resource` difference in one sentence?
- Can you name a trade-off you made in your capstone, and its cost?
- Can you say "I don't know" comfortably, and follow it with a plan?

81. Closing Notes

The four libraries in this book divide neatly:

- **NumPy** is about *memory* - contiguous typed buffers and the strides that make operations free.
- **Pandas** is about *labels* - alignment, grouping, and joining data that has meaning attached to its rows.
- **Flask** is about *boundaries* - the request/response contract, contexts, and where business logic should live.
- **Streamlit** is about *reruns* - one rule, and the caching and state tools that make it workable.

If you can explain those four sentences and then defend a project where each one mattered, you are prepared. The API surface is searchable; the mental models are not. Study the models.

Good luck.